PlussMa Distribution Documentation

PlussMa Contributors

Mar 04, 2026

CONTENTS

1 Quickstart **3**
- 1.1 Include Path . . . 3
- 1.2 Sample In One Line . . . 3
- 1.3 Keep Shape And Dtype . . . 3
- 1.4 Evaluate Densities And CDFs . . . 3
- 1.5 Typed Wrapper For Parameter-Bound Models . . . 4
- 1.6 Gaussian Mixture . . . 4
- 1.7 Intended Style . . . 4

2 API Reference **5**
- 2.1 `supports` . . . 5
- 2.2 `sample` . . . 5
- 2.3 `sample_like` . . . 6
- 2.4 `pdf` . . . 6
- 2.5 `logpdf` . . . 7
- 2.6 `cdf` . . . 7
- 2.7 `logcdf` . . . 7
- 2.8 `Object Wrappers` . . . 8
- 2.9 `mixture_normal` . . . 8
- 2.10 `mixture_normal_pdf` . . . 9
- 2.11 `mixture_normal_logpdf` . . . 9
- 2.12 `mixture_normal_cdf` . . . 10
- 2.13 `mixture_normal_logcdf` . . . 10

3 Object API Reference **13**
- 3.1 `Normal` . . . 13
- 3.2 `Bernoulli` . . . 14
- 3.3 `Binomial` . . . 15
- 3.4 `Poisson` . . . 16
- 3.5 `Geometric` . . . 17
- 3.6 `NegativeBinomial` . . . 18
- 3.7 `Gamma` . . . 19
- 3.8 `ChiSquare` . . . 20
- 3.9 `InverseGaussian` . . . 21
- 3.10 `Lognormal` . . . 22
- 3.11 `Uniform` . . . 23
- 3.12 `Exponential` . . . 24
- 3.13 `Laplace` . . . 25
- 3.14 `Logistic` . . . 26
- 3.15 `Rayleigh` . . . 27

3.16 NormalTrunc . . . 28
3.17 StudentT . . . 29
3.18 FDistribution . . . 30
3.19 Weibull . . . 31
3.20 Beta . . . 32
3.21 Skellam . . . 33
3.22 Nig . . . 34
3.23 Gh . . . 35
3.24 GaussianMixture . . . 36

4 Distribution Catalog **39**
4.1 normal . . . 39
4.2 lognormal . . . 40
4.3 uniform . . . 40
4.4 exponential . . . 41
4.5 bernoulli . . . 42
4.6 bessel . . . 42
4.7 beta . . . 43
4.8 binomial . . . 44
4.9 bradford . . . 45
4.10 burr . . . 45
4.11 cauchy . . . 46
4.12 chi . . . 47
4.13 chisquare . . . 47
4.14 chisqnoncentral . . . 48
4.15 cosine . . . 49
4.16 erlang . . . 49
4.17 extremevalue . . . 50
4.18 f . . . 51
4.19 fnoncentral . . . 51
4.20 laplace . . . 52
4.21 logistic . . . 53
4.22 geometric . . . 53
4.23 gamma . . . 54
4.24 gompertz . . . 55
4.25 halfcosine . . . 56
4.26 hyperbolicsecant . . . 56
4.27 hypergeometric . . . 57
4.28 inversegaussian . . . 58
4.29 maxwell . . . 58
4.30 nakagami . . . 59
4.31 negativebinomial . . . 60
4.32 normaltrunc . . . 60
4.33 planck . . . 61
4.34 poisson . . . 62
4.35 quadratic . . . 62
4.36 rademacher . . . 63
4.37 studentst . . . 64
4.38 triangular . . . 64
4.39 pareto . . . 65
4.40 lomax . . . 66
4.41 rayleigh . . . 66
4.42 rice . . . 67
4.43 semicircle . . . 68

4.44 skellam . . . 68
4.45 tukeylambda . . . 69
4.46 ushape . . . 70
4.47 vonmises . . . 70
4.48 wald . . . 71
4.49 weibull . . . 72
4.50 yule . . . 72
4.51 zeta . . . 73
4.52 gig . . . 74
4.53 gh . . . 75
4.54 nig . . . 75

5 Build Outputs **77**
5.1 Install Doc Dependencies . . . 77
5.2 Build HTML . . . 77
5.3 Build PDF . . . 77
5.4 Current Outputs . . . 78
5.5 Relevant Code Artifacts . . . 78
5.6 Validation Path . . . 78

The legacy Matlab `randraw.m` surface has been retired. Probability distributions now live in the C++ API as `pluss::distr`, available directly from:

```
#include <pluss/pluss.h>
```

This manual is organized around how users actually consume the layer:

- start with `quickstart` for the short-path workflow
- use `api_reference` when you need the function surface
- use `object_api_reference` when you want parameter-bound distribution objects
- use `distribution_catalog` when you need names, aliases, and parameter conventions
- use `build_outputs` when you want to build the docs or the example binary locally

CHAPTER

ONE

QUICKSTART

1.1 Include Path

```
#include <pluss/pluss.h>
```

The distribution layer is exposed as `pluss::distr`. No extra include is needed when PlussMa is part of the build.

1.2 Sample In One Line

```
auto x = pluss::distr::sample("normal", {0.0, 1.0}, {1024});
auto y = pluss::distr::sample("arcsin", {1024});
auto z = pluss::distr::sample("vonmises", {0.0, 3.0}, {1024});
```

1.3 Keep Shape And Dtype

```
auto like = pluss::zeros({2, 4},
    pluss::Config{}
        .with_dtype(pluss::ScalarType::Float32)
        .with_platform(pluss::pCPU)
        .with_pinned(true));

auto gamma_like = pluss::distr::sample_like("gamma", {2.0}, like);
auto signs_like = pluss::distr::sample_like("rademacher", like);
```

`sample_like(...)` preserves the visible tensor contract from `like`: shape, dtype, and platform. It also forwards the pinned flag into the sampling config, but the current runtime does not surface pinned-memory state through `is_pinned()` yet.

1.4 Evaluate Densities And CDFs

```
auto grid = pluss::make_series(std::vector<double>{-2.0, -1.0, 0.0, 1.0, 2.0}, {5});

auto normal_pdf = pluss::distr::pdf("normal", grid, {0.0, 1.0});
auto t_cdf = pluss::distr::cdf("t", grid, {5.0});
auto nig_logpdf = pluss::distr::logpdf("nig", grid, {2.0, 0.5, 0.0, 1.0});
```

1.5 Typed Wrapper For Parameter-Bound Models

```
pluss::distr::Normal normal{0.0, 1.0};
pluss::distr::Bernoulli bernoulli{0.35};
pluss::distr::Gamma gamma{2.0, 1.0, 0.0};
pluss::distr::Nig nig{2.0, 0.5, 0.0, 1.0};

auto normal_logpdf = normal.logpdf(grid);
auto draws = normal.sample({1024});
auto bern_logcdf = bernoulli.logcdf(pluss::make_series({0.0, 1.0}));
auto gamma_cgf = gamma.cgf(pluss::make_series({0.0, 0.5}));
auto nig_logtail = nig.logcdf(grid);
```

1.6 Gaussian Mixture

```
auto weights = pluss::make_series(std::vector<double>{0.65, 0.35}, {2});
auto means = pluss::make_series(std::vector<double>{-1.0, 2.0}, {2});
auto stddevs = pluss::make_series(std::vector<double>{0.5, 1.25}, {2});

auto mix = pluss::distr::mixture_normal(
    weights,
    means,
    stddevs,
    {4096},
    pluss::Config{}.with_dtype(pluss::ScalarType::Float64),
    23ULL,
    true);

pluss::distr::GaussianMixture gm{weights, means, stddevs};
auto gm_logtail = gm.logcdf(grid);
auto gm_cgf = gm.cgf(pluss::make_series({0.0, 0.1}));
```

The last argument switches the base normal generator from pseudo-random draws to Sobol-driven Gaussian draws.

1.7 Intended Style

The user-facing model is:

- keep everything under `pluss::distr`
- pass the distribution name plus a short parameter list
- use typed wrappers like `pluss::distr::Normal`, `Bernoulli`, `Binomial`, `Gamma`, `Uniform`, `Exponential`, `Skellam`, `Nig`, and `GaussianMixture` when a bound object is clearer
- let `sample_like` carry shape and dtype when you already have a target tensor
- stay on `Series` throughout instead of bouncing between helper wrappers

CHAPTER

TWO

API REFERENCE

2.1 supports

2.1.1 Signature

```cpp
bool pluss::distr::supports(std::string_view name) noexcept;
```

2.1.2 Notes

Checks whether the runtime resolver recognizes a family name or alias before you call `sample`, `pdf`, `logpdf`, or `cdf`.

2.1.3 Examples

```cpp
std::cout << pluss::distr::supports("gauss") << "\n";
std::cout << pluss::distr::supports("zipf") << "\n";
std::cout << pluss::distr::supports("not-a-dist") << "\n";
```

2.2 sample

2.2.1 Signature

```cpp
pluss::Series pluss::distr::sample(
    std::string_view name,
    std::span<const double> params,
    std::span<const std::int64_t> sizes,
    pluss::Config cfg = {},
    std::uint64_t seed = 0);
```

2.2.2 Notes

Samples a new `Series` from the chosen family. Use the parameter list required by that distribution, pass the output shape through `sizes`, and use `cfg` when you need to control dtype or platform.

2.2.3 Examples

```cpp
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
```

(continues on next page)

(continued from previous page)

```
auto draws = pluss::distr::sample("normal", {0.0, 1.0}, {2, 3}, cfg, 20260301ULL);
```

2.3 sample_like

2.3.1 Signature

```
pluss::Series pluss::distr::sample_like(
    std::string_view name,
    std::span<const double> params,
    const pluss::Series& like,
    std::uint64_t seed = 0);
```

2.3.2 Notes

Uses the same distribution parameters as `sample(...)`, but takes shape, dtype, and platform from `like`. This is the right helper when another `Series` already defines the tensor contract you want.

2.3.3 Examples

```
auto like = pluss::zeros(
    {2, 4},
    pluss::Config{}
        .with_dtype(pluss::ScalarType::Float32)
        .with_platform(pluss::pCPU));

auto out = pluss::distr::sample_like("gamma", {2.0}, like, 77ULL);
```

2.4 pdf

2.4.1 Signature

```
pluss::Series pluss::distr::pdf(
    std::string_view name,
    const pluss::Series& x,
    std::span<const double> params = {});
```

2.4.2 Notes

Evaluates the probability density function for continuous families or the mass function for discrete families at every point in `x`.

2.4.3 Examples

```
auto x = pluss::make_series({-1.0, 0.0, 1.0}).to(pluss::ScalarType::Float64);
auto dens = pluss::distr::pdf("normal", x, {0.0, 1.0});
```

2.5 logpdf

2.5.1 Signature

```
pluss::Series pluss::distr::logpdf(
    std::string_view name,
    const pluss::Series& x,
    std::span<const double> params = {});
```

2.5.2 Notes

Evaluates the natural log of the density or mass function. Use it when you are building log-likelihood objectives or want to avoid manually taking `log(pdf(...))`.

2.5.3 Examples

```
auto x = pluss::make_series({-1.0, 0.0, 1.0}).to(pluss::ScalarType::Float64);
auto logdens = pluss::distr::logpdf("nig", x, {2.0, 0.5, 0.0, 1.0});
```

2.6 cdf

2.6.1 Signature

```
pluss::Series pluss::distr::cdf(
    std::string_view name,
    const pluss::Series& x,
    std::span<const double> params = {});
```

2.6.2 Notes

Evaluates the cumulative distribution function at every point in `x`. Use it when your downstream logic is expressed in tail probabilities or threshold probabilities rather than densities.

2.6.3 Examples

```
auto x = pluss::make_series({-1.0, 0.0, 1.0}).to(pluss::ScalarType::Float64);
auto dist = pluss::distr::cdf("t", x, {5.0});
```

2.7 logcdf

2.7.1 Signature

```
pluss::Series pluss::distr::logcdf(
    std::string_view name,
    const pluss::Series& x,
    std::span<const double> params = {});
```

2.7.2 Notes

Evaluates the log of the cumulative distribution function directly. Use this instead of `cdf(...).log()` when you care about left-tail stability for extreme inputs.

2.7.3 Examples

```
auto x = pluss::make_series({-8.0, -2.0, 0.0}).to(pluss::ScalarType::Float64);
auto logtail = pluss::distr::logcdf("normal", x, {0.0, 1.0});
```

2.8 Object Wrappers

2.8.1 Signature

```
pluss::distr::Normal
pluss::distr::Bernoulli
pluss::distr::Binomial
pluss::distr::Gamma
pluss::distr::Uniform
pluss::distr::Exponential
pluss::distr::Skellam
pluss::distr::Nig
pluss::distr::GaussianMixture
```

2.8.2 Notes

The typed wrappers bind parameters into objects and then expose methods like `sample(...)`, `sample_like(...)`, `pdf(...)`, `logpdf(...)`, `cdf(...)`, `logcdf(...)`, and `cgf(...)`. Use them when the distribution itself is part of the model state and repeating raw parameter lists would make the code harder to read.

2.8.3 Examples

```
pluss::distr::Normal normal{0.0, 1.0};

auto draws = normal.sample({8});
auto logtail = normal.logcdf(pluss::make_series({-1.0, 0.0, 1.0}));
auto cumulants = normal.cgf(pluss::make_series({0.0, 0.25}));
```

2.9 mixture_normal

2.9.1 Signature

```
pluss::Series pluss::distr::mixture_normal(
    const pluss::Series& weights,
    const pluss::Series& means,
    const pluss::Series& stddevs,
    std::span<const std::int64_t> sizes,
    pluss::Config cfg = {},
    std::uint64_t seed = 0,
    bool use_sobol = false);
```

2.9.2 Notes

Samples from a Gaussian mixture defined by component weights, means, and standard deviations. Set `use_sobol=true` when you want Sobol-driven quasi-random normal draws instead of pseudo-random draws.

2.9.3 Examples

```
auto cfg = pluss::Config{}.with_dtype(pluss::ScalarType::Float64).with_
↪platform(pluss::pCPU);
auto weights = pluss::make_series({0.65, 0.35}, cfg);
auto means = pluss::make_series({-1.0, 2.0}, cfg);
auto stddevs = pluss::make_series({0.5, 1.25}, cfg);

auto mix = pluss::distr::mixture_normal(weights, means, stddevs, {8}, cfg, 101ULL, true);
```

2.10 mixture_normal_pdf

2.10.1 Signature

```
pluss::Series pluss::distr::mixture_normal_pdf(
    const pluss::Series& x,
    const pluss::Series& weights,
    const pluss::Series& means,
    const pluss::Series& stddevs);
```

2.10.2 Notes

Evaluates the density of a Gaussian mixture that you have already parameterized as component `Series` values.

2.10.3 Examples

```
auto x = pluss::make_series({-2.0, -0.5, 1.0, 3.0}).to(pluss::ScalarType::Float64);
auto weights = pluss::make_series({0.65, 0.35}).to(pluss::ScalarType::Float64);
auto means = pluss::make_series({-1.0, 2.0}).to(pluss::ScalarType::Float64);
auto stddevs = pluss::make_series({0.5, 1.25}).to(pluss::ScalarType::Float64);

auto dens = pluss::distr::mixture_normal_pdf(x, weights, means, stddevs);
```

2.11 mixture_normal_logpdf

2.11.1 Signature

```
pluss::Series pluss::distr::mixture_normal_logpdf(
    const pluss::Series& x,
    const pluss::Series& weights,
    const pluss::Series& means,
    const pluss::Series& stddevs);
```

2.11.2 Notes

Evaluates the log-density of the same Gaussian mixture parameterization used by `mixture_normal_pdf(...)`.

2.11.3 Examples

```
auto x = pluss::make_series({-2.0, -0.5, 1.0, 3.0}).to(pluss::ScalarType::Float64);
auto weights = pluss::make_series({0.65, 0.35}).to(pluss::ScalarType::Float64);
auto means = pluss::make_series({-1.0, 2.0}).to(pluss::ScalarType::Float64);
auto stddevs = pluss::make_series({0.5, 1.25}).to(pluss::ScalarType::Float64);

auto logdens = pluss::distr::mixture_normal_logpdf(x, weights, means, stddevs);
```

2.12 mixture_normal_cdf

2.12.1 Signature

```
pluss::Series pluss::distr::mixture_normal_cdf(
    const pluss::Series& x,
    const pluss::Series& weights,
    const pluss::Series& means,
    const pluss::Series& stddevs);
```

2.12.2 Notes

Evaluates the mixture CDF using the same component parameterization as the other Gaussian-mixture helpers.

2.12.3 Examples

```
auto x = pluss::make_series({-2.0, -0.5, 1.0, 3.0}).to(pluss::ScalarType::Float64);
auto weights = pluss::make_series({0.65, 0.35}).to(pluss::ScalarType::Float64);
auto means = pluss::make_series({-1.0, 2.0}).to(pluss::ScalarType::Float64);
auto stddevs = pluss::make_series({0.5, 1.25}).to(pluss::ScalarType::Float64);

auto dist = pluss::distr::mixture_normal_cdf(x, weights, means, stddevs);
```

2.13 mixture_normal_logcdf

2.13.1 Signature

```
pluss::Series pluss::distr::mixture_normal_logcdf(
    const pluss::Series& x,
    const pluss::Series& weights,
    const pluss::Series& means,
    const pluss::Series& stddevs);
```

2.13.2 Notes

Evaluates the log of the Gaussian-mixture CDF directly. This is the stable choice when you need extreme left-tail mixture probabilities without taking `log(mixture_normal_cdf(...))` yourself.

2.13.3 Examples

```
auto x = pluss::make_series({-6.0, -1.0, 0.0}).to(pluss::ScalarType::Float64);
auto weights = pluss::make_series({0.65, 0.35}).to(pluss::ScalarType::Float64);
auto means = pluss::make_series({-1.0, 2.0}).to(pluss::ScalarType::Float64);
auto stddevs = pluss::make_series({0.5, 1.25}).to(pluss::ScalarType::Float64);

auto logtail = pluss::distr::mixture_normal_logcdf(x, weights, means, stddevs);
```

CHAPTER

THREE

OBJECT API REFERENCE

The object wrappers bind distribution parameters once, then expose a method surface that stays on `Series` throughout:

- `sample(...)`
- `sample_like(...)`
- `pdf(...)`
- `logpdf(...)`
- `cdf(...)`
- `logcdf(...)`
- `cgf(...)`
- `cumulant(...)`

Use them when the distribution itself is part of the model state and repeated calls should not keep re-spelling the parameter list.

All `Output` blocks below were verified from the compiled demo `distributions_objects_demo.cpp` using the fixed seed `20260303ULL` on CPU. If you change dtype, platform, or seed, sampled values will change.

3.1 Normal

3.1.1 Signature

```
pluss::distr::Normal normal{mean, stddev};

normal.sample(sizes, cfg = {}, seed = 0);
normal.sample_like(like, seed = 0);
normal.pdf(x);
normal.logpdf(x);
normal.cdf(x);
normal.logcdf(x);
normal.cgf(t);
normal.cumulant(order);
```

3.1.2 Notes

Represents a Gaussian with fixed `mean` and `stddev`. `cumulant(1)` returns the mean, `cumulant(2)` returns the variance, and higher orders are zero, so it is a good baseline for cumulant-driven code paths.

3.1.3 Examples

```
pluss::distr::Normal normal{0.0, 1.0};
auto x = pluss::make_series({-1.0, 0.0, 0.5, 1.0});
auto like = pluss::zeros({4},
    pluss::Config{}.with_dtype(pluss::ScalarType::Float32).with_platform(pluss::pCPU));
auto t = pluss::make_series({-0.25, 0.0, 0.25});

auto dens = normal.pdf(x);
auto logdens = normal.logpdf(x);
auto dist = normal.cdf(x);
auto logtail = normal.logcdf(x);
auto cumulants = normal.cgf(t);
auto k1 = normal.cumulant(1);
auto k2 = normal.cumulant(2);
auto draws = normal.sample({4}, pluss::Config{}.with_dtype(pluss::ScalarType::Float64),␣
↪20260303ULL);
auto like_draws = normal.sample_like(like, 20260303ULL);
```

Output

```
pdf(x):      [0.241971, 0.398942, 0.352065, 0.241971]
logpdf(x):   [-1.418939, -0.918939, -1.043939, -1.418939]
cdf(x):      [0.158655, 0.500000, 0.691462, 0.841345]
logcdf(x):   [-1.841022, -0.693147, -0.368946, -0.172754]
cgf(t):      [0.031250, 0.000000, 0.031250]
cumulants:   [0, 1, 0, 0]
sample({4}): [0.912964, -0.916896, -1.181625, 0.307058]
sample_like: [0.912964, -0.916896, -1.181625, 0.307058] dtype=11
```

3.2 Bernoulli

3.2.1 Signature

```
pluss::distr::Bernoulli bernoulli{p};

bernoulli.sample(sizes, cfg = {}, seed = 0);
bernoulli.sample_like(like, seed = 0);
bernoulli.pdf(x);
bernoulli.logpdf(x);
bernoulli.cdf(x);
bernoulli.logcdf(x);
bernoulli.cgf(t);
bernoulli.cumulant(order);
```

3.2.2 Notes

Binds a success probability once and exposes the full 0/1 event model. `cumulant(...)` is implemented from Bernoulli raw moments, so higher orders remain available.

3.2.3 Examples

```
pluss::distr::Bernoulli bernoulli{0.25};
auto x = pluss::make_series({0.0, 1.0, 2.0, 3.0});
auto like = pluss::zeros({4},
    pluss::Config{}.with_dtype(pluss::ScalarType::Float32).with_platform(pluss::pCPU));
auto t = pluss::make_series({-0.25, 0.0, 0.25});

auto pmf = bernoulli.pdf(x);
auto logprob = bernoulli.logpdf(x);
auto dist = bernoulli.cdf(x);
auto logtail = bernoulli.logcdf(x);
auto cgf = bernoulli.cgf(t);
auto k1 = bernoulli.cumulant(1);
auto k2 = bernoulli.cumulant(2);
auto draws = bernoulli.sample({4}, pluss::Config{}.with_
↪dtype(pluss::ScalarType::Float64), 20260303ULL);
auto like_draws = bernoulli.sample_like(like, 20260303ULL);
```

Output

```
pdf(x):      [0.750000, 0.250000, 0.000000, 0.000000]
logpdf(x):   [-0.287682, -1.386294, -inf, -inf]
cdf(x):      [0.750000, 1.000000, 1.000000, 1.000000]
logcdf(x):   [-0.287682, 0.000000, 0.000000, 0.000000]
cgf(t):      [-0.056888, 0.000000, 0.068599]
cumulants:   [0.25, 0.1875, 0.09375, -0.0234375]
sample({4}): [0.000000, 0.000000, 0.000000, 0.000000]
sample_like: [0.000000, 0.000000, 0.000000, 0.000000] dtype=11
```

3.3 Binomial

3.3.1 Signature

```
pluss::distr::Binomial binomial{count, p};

binomial.sample(sizes, cfg = {}, seed = 0);
binomial.sample_like(like, seed = 0);
binomial.pdf(x);
binomial.logpdf(x);
binomial.cdf(x);
binomial.logcdf(x);
binomial.cgf(t);
binomial.cumulant(order);
```

3.3.2 Notes

Models the number of successes across `count` Bernoulli trials with success rate `p`. `cumulant(order)` reuses the Bernoulli cumulants and scales them by `count`.

3.3.3 Examples

```
pluss::distr::Binomial binomial{4, 0.25};
auto x = pluss::make_series({0.0, 1.0, 2.0, 3.0});
auto like = pluss::zeros({4},
    pluss::Config{}.with_dtype(pluss::ScalarType::Float32).with_platform(pluss::pCPU));
auto t = pluss::make_series({-0.25, 0.0, 0.25});

auto pmf = binomial.pdf(x);
auto logprob = binomial.logpdf(x);
auto dist = binomial.cdf(x);
auto logtail = binomial.logcdf(x);
auto cgf = binomial.cgf(t);
auto k1 = binomial.cumulant(1);
auto k2 = binomial.cumulant(2);
auto draws = binomial.sample({4}, pluss::Config{}.with_dtype(pluss::ScalarType::Float64),
↪ 20260303ULL);
auto like_draws = binomial.sample_like(like, 20260303ULL);
```

Output

```
pdf(x):      [0.316406, 0.421875, 0.210937, 0.046875]
logpdf(x):   [-1.150728, -0.863046, -1.556193, -3.060271]
cdf(x):      [0.316406, 0.738281, 0.949219, 0.996094]
logcdf(x):   [-1.150728, -0.303430, -0.052116, -0.003914]
cgf(t):      [-0.227551, 0.000000, 0.274395]
cumulants:   [1, 0.75, 0.375, -0.09375]
sample({4}): [3.000000, 2.000000, 0.000000, 3.000000]
sample_like: [3.000000, 2.000000, 0.000000, 3.000000] dtype=11
```

3.4 Poisson

3.4.1 Signature

```
pluss::distr::Poisson poisson{lambda};

poisson.sample(sizes, cfg = {}, seed = 0);
poisson.sample_like(like, seed = 0);
poisson.pdf(x);
poisson.logpdf(x);
poisson.cdf(x);
poisson.logcdf(x);
poisson.cgf(t);
poisson.cumulant(order);
```

3.4.2 Notes

Wraps the Poisson count model for a fixed event rate `lambda`. Every cumulant equals `lambda`, which makes this object convenient in count models that consume cumulants directly.

3.4.3 Examples

```
pluss::distr::Poisson poisson{2.5};
auto x = pluss::make_series({0.0, 1.0, 2.0, 3.0});
auto like = pluss::zeros({4},
    pluss::Config{}.with_dtype(pluss::ScalarType::Float32).with_platform(pluss::pCPU));
auto t = pluss::make_series({-0.25, 0.0, 0.25});

auto pmf = poisson.pdf(x);
auto logprob = poisson.logpdf(x);
auto dist = poisson.cdf(x);
auto logtail = poisson.logcdf(x);
auto cgf = poisson.cgf(t);
auto k1 = poisson.cumulant(1);
auto k2 = poisson.cumulant(2);
auto draws = poisson.sample({4}, pluss::Config{}.with_dtype(pluss::ScalarType::Float64),␣
↪20260303ULL);
auto like_draws = poisson.sample_like(like, 20260303ULL);
```

Output

```
pdf(x):      [0.082085, 0.205212, 0.256516, 0.213763]
logpdf(x):   [-2.500000, -1.583709, -1.360566, -1.542887]
cdf(x):      [0.082085, 0.287297, 0.543813, 0.757576]
logcdf(x):   [-2.500000, -1.247237, -0.609150, -0.277631]
cgf(t):      [-0.552998, 0.000000, 0.710064]
cumulants:   [2.5, 2.5, 2.5, 2.5]
sample({4}): [7.000000, 0.000000, 4.000000, 4.000000]
sample_like: [7.000000, 0.000000, 4.000000, 4.000000] dtype=11
```

3.5 Geometric

3.5.1 Signature

```
pluss::distr::Geometric geometric{p};

geometric.sample(sizes, cfg = {}, seed = 0);
geometric.sample_like(like, seed = 0);
geometric.pdf(x);
geometric.logpdf(x);
geometric.cdf(x);
geometric.logcdf(x);
geometric.cgf(t);
geometric.cumulant(order);
```

3.5.2 Notes

Uses the success-probability parameterization of the geometric law. The demo evaluates it on positive counts because support starts at 1 in this implementation.

3.5.3 Examples

```
pluss::distr::Geometric geometric{0.25};
auto x = pluss::make_series({1.0, 2.0, 3.0, 4.0});
auto like = pluss::zeros({4},
    pluss::Config{}.with_dtype(pluss::ScalarType::Float32).with_platform(pluss::pCPU));
auto t = pluss::make_series({-0.25, 0.0, 0.25});

auto pmf = geometric.pdf(x);
auto logprob = geometric.logpdf(x);
auto dist = geometric.cdf(x);
auto logtail = geometric.logcdf(x);
auto cgf = geometric.cgf(t);
auto k1 = geometric.cumulant(1);
auto k2 = geometric.cumulant(2);
auto draws = geometric.sample({4}, pluss::Config{}.with_
↪dtype(pluss::ScalarType::Float64), 20260303ULL);
auto like_draws = geometric.sample_like(like, 20260303ULL);
```

Output

```
pdf(x):      [0.250000, 0.187500, 0.140625, 0.105469]
logpdf(x):   [-1.386294, -1.673976, -1.961659, -2.249341]
cdf(x):      [0.250000, 0.437500, 0.578125, 0.683594]
logcdf(x):   [-1.386294, -0.826679, -0.547965, -0.380391]
cgf(t):      [-0.758983, 0.000000, 2.161058]
cumulants:   [4, 12, 84, 876]
sample({4}): [2.000000, 8.000000, 3.000000, 3.000000]
sample_like: [2.000000, 8.000000, 3.000000, 3.000000] dtype=11
```

3.6 NegativeBinomial

3.6.1 Signature

```
pluss::distr::NegativeBinomial negative_binomial{r, p};

negative_binomial.sample(sizes, cfg = {}, seed = 0);
negative_binomial.sample_like(like, seed = 0);
negative_binomial.pdf(x);
negative_binomial.logpdf(x);
negative_binomial.cdf(x);
negative_binomial.logcdf(x);
negative_binomial.cgf(t);
negative_binomial.cumulant(order);
```

3.6.2 Notes

Represents the negative-binomial count law with dispersion-like parameter `r` and success probability `p`. Use it when Poisson variance is too restrictive.

3.6.3 Examples

```
pluss::distr::NegativeBinomial negative_binomial{3.0, 0.4};
auto x = pluss::make_series({0.0, 1.0, 2.0, 3.0});
auto like = pluss::zeros({4},
    pluss::Config{}.with_dtype(pluss::ScalarType::Float32).with_platform(pluss::pCPU));
auto t = pluss::make_series({-0.25, 0.0, 0.25});

auto pmf = negative_binomial.pdf(x);
auto logprob = negative_binomial.logpdf(x);
auto dist = negative_binomial.cdf(x);
auto logtail = negative_binomial.logcdf(x);
auto cgf = negative_binomial.cgf(t);
auto k1 = negative_binomial.cumulant(1);
auto k2 = negative_binomial.cumulant(2);
auto draws = negative_binomial.sample({4}, pluss::Config{}.with_
↪dtype(pluss::ScalarType::Float64), 20260303ULL);
auto like_draws = negative_binomial.sample_like(like, 20260303ULL);
```

Output

```
pdf(x):      [0.064000, 0.115200, 0.138240, 0.138240]
logpdf(x):   [-2.748872, -2.161086, -1.978764, -1.978764]
cdf(x):      [0.064000, 0.179200, 0.317440, 0.455680]
logcdf(x):   [-2.748872, -1.719253, -1.147466, -0.785964]
cgf(t):      [-0.859592, 0.000000, 1.665577]
cumulants:   [4.5, 11.25, 45, 264.375]
sample({4}): [4.000000, 8.000000, 1.000000, 9.000000]
sample_like: [4.000000, 8.000000, 1.000000, 9.000000] dtype=11
```

3.7 Gamma

3.7.1 Signature

```
pluss::distr::Gamma gamma{shape, scale, location = 0.0};

gamma.sample(sizes, cfg = {}, seed = 0);
gamma.sample_like(like, seed = 0);
gamma.pdf(x);
gamma.logpdf(x);
gamma.cdf(x);
gamma.logcdf(x);
gamma.cgf(t);
gamma.cumulant(order);
```

3.7.2 Notes

Encodes the shifted gamma family directly as object state. `cgf(...)` enforces the domain restriction `t < 1 / scale`, and `cumulant(order)` is available for every positive order.

3.7.3 Examples

```
pluss::distr::Gamma gamma{2.0, 1.0, 0.0};
auto x = pluss::make_series({0.5, 1.5, 2.0, 2.5});
auto like = pluss::zeros({4},
    pluss::Config{}.with_dtype(pluss::ScalarType::Float32).with_platform(pluss::pCPU));
auto t = pluss::make_series({-0.25, 0.0, 0.25});

auto dens = gamma.pdf(x);
auto logdens = gamma.logpdf(x);
auto dist = gamma.cdf(x);
auto logtail = gamma.logcdf(x);
auto cgf = gamma.cgf(t);
auto k1 = gamma.cumulant(1);
auto k2 = gamma.cumulant(2);
auto draws = gamma.sample({4}, pluss::Config{}.with_dtype(pluss::ScalarType::Float64),␣
↪20260303ULL);
auto like_draws = gamma.sample_like(like, 20260303ULL);
```

Output

```
pdf(x):      [0.303265, 0.334695, 0.270671, 0.205212]
logpdf(x):   [-1.193147, -1.094535, -1.306853, -1.583709]
cdf(x):      [0.090204, 0.442175, 0.593994, 0.712703]
logcdf(x):   [-2.405681, -0.816050, -0.520886, -0.338691]
cgf(t):      [-0.446287, 0.000000, 0.575364]
cumulants:   [2, 2, 4, 12]
sample({4}): [1.833701, 1.832495, 2.016090, 1.644138]
sample_like: [1.833701, 1.832494, 2.016090, 1.644138] dtype=11
```

3.8 ChiSquare

3.8.1 Signature

```
pluss::distr::ChiSquare chi_square{degrees_of_freedom};

chi_square.sample(sizes, cfg = {}, seed = 0);
chi_square.sample_like(like, seed = 0);
chi_square.pdf(x);
chi_square.logpdf(x);
chi_square.cdf(x);
chi_square.logcdf(x);
chi_square.cgf(t);
chi_square.cumulant(order);
```

3.8.2 Notes

Wraps the chi-square family in terms of degrees of freedom. Its cumulants grow quickly with order, so it is useful when you need variance and tail-heavy shape diagnostics from the same object.

3.8.3 Examples

```
pluss::distr::ChiSquare chi_square{5};
auto x = pluss::make_series({1.0, 2.0, 2.5, 3.0});
auto like = pluss::zeros({4},
    pluss::Config{}.with_dtype(pluss::ScalarType::Float32).with_platform(pluss::pCPU));
auto t = pluss::make_series({-0.25, 0.0, 0.25});

auto dens = chi_square.pdf(x);
auto logdens = chi_square.logpdf(x);
auto dist = chi_square.cdf(x);
auto logtail = chi_square.logcdf(x);
auto cgf = chi_square.cgf(t);
auto k1 = chi_square.cumulant(1);
auto k2 = chi_square.cumulant(2);
auto draws = chi_square.sample({4}, pluss::Config{}.with_
↪dtype(pluss::ScalarType::Float64), 20260303ULL);
auto like_draws = chi_square.sample_like(like, 20260303ULL);
```

Output

```
pdf(x):      [0.080657, 0.138369, 0.150602, 0.154180]
logpdf(x):   [-2.517551, -1.977830, -1.893115, -1.869632]
cdf(x):      [0.037434, 0.150855, 0.223505, 0.300014]
logcdf(x):   [-3.285170, -1.891436, -1.498322, -1.203926]
cgf(t):      [-1.013663, 0.000000, 1.732868]
cumulants:   [5, 10, 40, 240]
sample({4}): [4.890656, 4.887920, 5.304276, 4.460768]
sample_like: [4.890656, 4.887920, 5.304276, 4.460768] dtype=11
```

3.9 InverseGaussian

3.9.1 Signature

```
pluss::distr::InverseGaussian inverse_gaussian{theta, chi};

inverse_gaussian.sample(sizes, cfg = {}, seed = 0);
inverse_gaussian.sample_like(like, seed = 0);
inverse_gaussian.pdf(x);
inverse_gaussian.logpdf(x);
inverse_gaussian.cdf(x);
inverse_gaussian.logcdf(x);
inverse_gaussian.cgf(t);
inverse_gaussian.cumulant(order);
```

3.9.2 Notes

Represents the inverse-Gaussian family with mean-like parameter `theta` and shape parameter `chi`. The wrapper exposes exact lower-order cumulants, which is useful in normal-variance-mixture workflows.

3.9.3 Examples

```
pluss::distr::InverseGaussian inverse_gaussian{1.5, 3.0};
auto x = pluss::make_series({0.5, 1.5, 2.0, 2.5});
auto like = pluss::zeros({4},
    pluss::Config{}.with_dtype(pluss::ScalarType::Float32).with_platform(pluss::pCPU));
auto t = pluss::make_series({-0.25, 0.0, 0.25});

auto dens = inverse_gaussian.pdf(x);
auto logdens = inverse_gaussian.logpdf(x);
auto dist = inverse_gaussian.cdf(x);
auto logtail = inverse_gaussian.logcdf(x);
auto cgf = inverse_gaussian.cgf(t);
auto k1 = inverse_gaussian.cumulant(1);
auto k2 = inverse_gaussian.cumulant(2);
auto draws = inverse_gaussian.sample({4}, pluss::Config{}.with_
↪dtype(pluss::ScalarType::Float64), 20260303ULL);
auto like_draws = inverse_gaussian.sample_like(like, 20260303ULL);
```

Output

```
pdf(x):      [0.515177, 0.376126, 0.224768, 0.133890]
logpdf(x):   [-0.663245, -0.977830, -1.492686, -2.010735]
cdf(x):      [0.081014, 0.627698, 0.774932, 0.862588]
logcdf(x):   [-2.513133, -0.465696, -0.254980, -0.147819]
cgf(t):      [-0.345208, 0.000000, 0.418861]
cumulants:   [1.5, 1.125, 2.53125, 9.49219]
sample({4}): [1.646021, 1.509892, 0.347783, 1.907487]
sample_like: [1.646021, 1.509892, 0.347783, 1.907487] dtype=11
```

3.10 Lognormal

3.10.1 Signature

```
pluss::distr::Lognormal lognormal{mean_log, stddev_log};

lognormal.sample(sizes, cfg = {}, seed = 0);
lognormal.sample_like(like, seed = 0);
lognormal.pdf(x);
lognormal.logpdf(x);
lognormal.cdf(x);
lognormal.logcdf(x);
lognormal.cgf(t);
lognormal.cumulant(order);
```

3.10.2 Notes

Stores the normal parameters in log-space and evaluates on the positive real line. The CGF is only documented on the nonpositive grid used in the demo because the ordinary moment generating function diverges for positive arguments.

3.10.3 Examples

```
pluss::distr::Lognormal lognormal{0.1, 0.4};
auto x = pluss::make_series({0.5, 1.5, 2.0, 2.5});
auto like = pluss::zeros({4},
    pluss::Config{}.with_dtype(pluss::ScalarType::Float32).with_platform(pluss::pCPU));
auto t = pluss::make_series({-0.5, -0.25, 0.0});

auto dens = lognormal.pdf(x);
auto logdens = lognormal.logpdf(x);
auto dist = lognormal.cdf(x);
auto logtail = lognormal.logcdf(x);
auto cgf = lognormal.cgf(t);
auto k1 = lognormal.cumulant(1);
auto k2 = lognormal.cumulant(2);
auto draws = lognormal.sample({4}, pluss::Config{}.with_
↪dtype(pluss::ScalarType::Float64), 20260303ULL);
auto like_draws = lognormal.sample_like(like, 20260303ULL);
```

Output

```
pdf(x):      [0.279324, 0.496733, 0.166087, 0.049726]
logpdf(x):   [-1.275383, -0.699703, -1.795244, -3.001222]
cdf(x):      [0.023691, 0.777466, 0.930945, 0.979361]
logcdf(x):   [-3.742656, -0.251716, -0.071555, -0.020855]
cgf(t):      [-0.570493, -0.291929, 0.000000]
cumulants:   [1.19722, 0.248698, 0.16395, 0.201634]
sample({4}): [1.592310, 0.765858, 0.688906, 1.249600]
sample_like: [1.592310, 0.765858, 0.688906, 1.249600] dtype=11
```

3.11 Uniform

3.11.1 Signature

```
pluss::distr::Uniform uniform{low, high};

uniform.sample(sizes, cfg = {}, seed = 0);
uniform.sample_like(like, seed = 0);
uniform.pdf(x);
uniform.logpdf(x);
uniform.cdf(x);
uniform.logcdf(x);
uniform.cgf(t);
uniform.cumulant(order);
```

3.11.2 Notes

Represents a bounded continuous law on `[low, high]`. Higher-order cumulants come from the interval raw moments, so the object remains useful beyond simple sampling.

3.11.3 Examples

```
pluss::distr::Uniform uniform{0.0, 1.0};
auto x = pluss::make_series({0.0, 0.5, 0.75, 1.0});
auto like = pluss::zeros({4},
    pluss::Config{}.with_dtype(pluss::ScalarType::Float32).with_platform(pluss::pCPU));
auto t = pluss::make_series({-0.25, 0.0, 0.25});

auto dens = uniform.pdf(x);
auto logdens = uniform.logpdf(x);
auto dist = uniform.cdf(x);
auto logtail = uniform.logcdf(x);
auto cgf = uniform.cgf(t);
auto k1 = uniform.cumulant(1);
auto k2 = uniform.cumulant(2);
auto draws = uniform.sample({4}, pluss::Config{}.with_dtype(pluss::ScalarType::Float64),␣
↪20260303ULL);
auto like_draws = uniform.sample_like(like, 20260303ULL);
```

Output

```
pdf(x):      [1.000000, 1.000000, 1.000000, 1.000000]
logpdf(x):   [-0.000000, -0.000000, -0.000000, -0.000000]
cdf(x):      [0.000000, 0.500000, 0.750000, 1.000000]
logcdf(x):   [-inf, -0.693147, -0.287682, 0.000000]
cgf(t):      [-0.122397, 0.000000, 0.127603]
cumulants:   [0.5, 0.0833333, 2.77556e-17, -0.00833333]
sample({4}): [0.432964, 0.874658, 0.474611, 0.459537]
sample_like: [0.432964, 0.874658, 0.474611, 0.459537] dtype=11
```

3.12 Exponential

3.12.1 Signature

```
pluss::distr::Exponential exponential{rate};

exponential.sample(sizes, cfg = {}, seed = 0);
exponential.sample_like(like, seed = 0);
exponential.pdf(x);
exponential.logpdf(x);
exponential.cdf(x);
exponential.logcdf(x);
exponential.cgf(t);
exponential.cumulant(order);
```

3.12.2 Notes

Uses the rate parameterization. `cgf(...)` enforces `t < rate`, and `cumulant(order)` follows the closed-form exponential cumulants.

3.12.3 Examples

```
pluss::distr::Exponential exponential{2.0};
auto x = pluss::make_series({0.5, 1.5, 2.0, 2.5});
auto like = pluss::zeros({4},
    pluss::Config{}.with_dtype(pluss::ScalarType::Float32).with_platform(pluss::pCPU));
auto t = pluss::make_series({-0.25, 0.0, 0.25});

auto dens = exponential.pdf(x);
auto logdens = exponential.logpdf(x);
auto dist = exponential.cdf(x);
auto logtail = exponential.logcdf(x);
auto cgf = exponential.cgf(t);
auto k1 = exponential.cumulant(1);
auto k2 = exponential.cumulant(2);
auto draws = exponential.sample({4}, pluss::Config{}.with_
↪dtype(pluss::ScalarType::Float64), 20260303ULL);
auto like_draws = exponential.sample_like(like, 20260303ULL);
```

Output

```
pdf(x):      [0.735759, 0.099574, 0.036631, 0.013476]
logpdf(x):   [-0.306853, -2.306853, -3.306853, -4.306853]
cdf(x):      [0.632121, 0.950213, 0.981684, 0.993262]
logcdf(x):   [-0.458675, -0.051069, -0.018485, -0.006761]
cgf(t):      [-0.117783, 0.000000, 0.133531]
cumulants:   [0.5, 0.25, 0.25, 0.375]
sample({4}): [0.418551, 0.066961, 0.372630, 0.388768]
sample_like: [0.418551, 0.066961, 0.372630, 0.388768] dtype=11
```

3.13 Laplace

3.13.1 Signature

```
pluss::distr::Laplace laplace{location, scale};

laplace.sample(sizes, cfg = {}, seed = 0);
laplace.sample_like(like, seed = 0);
laplace.pdf(x);
laplace.logpdf(x);
laplace.cdf(x);
laplace.logcdf(x);
laplace.cgf(t);
laplace.cumulant(order);
```

3.13.2 Notes

Represents the double-exponential law centered at `location`. Odd cumulants above the mean vanish, which makes the wrapper a clean symmetric heavy-tail reference.

3.13.3 Examples

```
pluss::distr::Laplace laplace{0.0, 0.5};
auto x = pluss::make_series({-1.0, 0.0, 0.5, 1.0});
auto like = pluss::zeros({4},
    pluss::Config{}.with_dtype(pluss::ScalarType::Float32).with_platform(pluss::pCPU));
auto t = pluss::make_series({-0.25, 0.0, 0.25});

auto dens = laplace.pdf(x);
auto logdens = laplace.logpdf(x);
auto dist = laplace.cdf(x);
auto logtail = laplace.logcdf(x);
auto cgf = laplace.cgf(t);
auto k1 = laplace.cumulant(1);
auto k2 = laplace.cumulant(2);
auto draws = laplace.sample({4}, pluss::Config{}.with_dtype(pluss::ScalarType::Float64),␣
↪20260303ULL);
auto like_draws = laplace.sample_like(like, 20260303ULL);
```

Output

```
pdf(x):      [0.135335, 1.000000, 0.367879, 0.135335]
logpdf(x):   [-2.000000, -0.000000, -1.000000, -2.000000]
cdf(x):      [0.067668, 0.500000, 0.816060, 0.932332]
logcdf(x):   [-2.693147, -0.693147, -0.203267, -0.070066]
cgf(t):      [0.015748, 0.000000, 0.015748]
cumulants:   [0, 0.5, 0, 0.75]
sample({4}): [-0.071977, 0.691781, -0.026057, -0.042195]
sample_like: [-0.071977, 0.691781, -0.026057, -0.042195] dtype=11
```

3.14 Logistic

3.14.1 Signature

```
pluss::distr::Logistic logistic{location, scale};

logistic.sample(sizes, cfg = {}, seed = 0);
logistic.sample_like(like, seed = 0);
logistic.pdf(x);
logistic.logpdf(x);
logistic.cdf(x);
logistic.logcdf(x);
logistic.cgf(t);
logistic.cumulant(order);
```

3.14.2 Notes

Wraps the logistic family with explicit location and scale. It is a good alternative to `Normal` when you want similar symmetry but heavier shoulders and tails.

3.14.3 Examples

```
pluss::distr::Logistic logistic{0.0, 0.75};
auto x = pluss::make_series({-1.0, 0.0, 0.5, 1.0});
auto like = pluss::zeros({4},
    pluss::Config{}.with_dtype(pluss::ScalarType::Float32).with_platform(pluss::pCPU));
auto t = pluss::make_series({-0.25, 0.0, 0.25});

auto dens = logistic.pdf(x);
auto logdens = logistic.logpdf(x);
auto dist = logistic.cdf(x);
auto logtail = logistic.logcdf(x);
auto cgf = logistic.cgf(t);
auto k1 = logistic.cumulant(1);
auto k2 = logistic.cumulant(2);
auto draws = logistic.sample({4}, pluss::Config{}.with_dtype(pluss::ScalarType::Float64),
↪ 20260303ULL);
auto like_draws = logistic.sample_like(like, 20260303ULL);
```

Output

```
pdf(x):      [0.220121, 0.333333, 0.298877, 0.220121]
logpdf(x):   [-1.513576, -1.098612, -1.207725, -1.513576]
cdf(x):      [0.208609, 0.500000, 0.660756, 0.791391]
logcdf(x):   [-1.567296, -0.693147, -0.414370, -0.233963]
cgf(t):      [0.058514, 0.000000, 0.058514]
cumulants:   [0, 1.85055, 0, 4.10945]
sample({4}): [-0.202327, 1.457090, -0.076234, -0.121656]
sample_like: [-0.202327, 1.457090, -0.076234, -0.121656] dtype=11
```

3.15 Rayleigh

3.15.1 Signature

```
pluss::distr::Rayleigh rayleigh{sigma};

rayleigh.sample(sizes, cfg = {}, seed = 0);
rayleigh.sample_like(like, seed = 0);
rayleigh.pdf(x);
rayleigh.logpdf(x);
rayleigh.cdf(x);
rayleigh.logcdf(x);
rayleigh.cgf(t);
rayleigh.cumulant(order);
```

3.15.2 Notes

Represents the magnitude distribution induced by two independent centered Gaussians. Use it for strictly positive radial data with a single scale parameter.

3.15.3 Examples

```
pluss::distr::Rayleigh rayleigh{1.0};
auto x = pluss::make_series({0.5, 1.5, 2.0, 2.5});
auto like = pluss::zeros({4},
    pluss::Config{}.with_dtype(pluss::ScalarType::Float32).with_platform(pluss::pCPU));
auto t = pluss::make_series({-0.25, 0.0, 0.25});

auto dens = rayleigh.pdf(x);
auto logdens = rayleigh.logpdf(x);
auto dist = rayleigh.cdf(x);
auto logtail = rayleigh.logcdf(x);
auto cgf = rayleigh.cgf(t);
auto k1 = rayleigh.cumulant(1);
auto k2 = rayleigh.cumulant(2);
auto draws = rayleigh.sample({4}, pluss::Config{}.with_dtype(pluss::ScalarType::Float64),
↪ 20260303ULL);
auto like_draws = rayleigh.sample_like(like, 20260303ULL);
```

Output

```
pdf(x):      [0.441248, 0.486979, 0.270671, 0.109842]
logpdf(x):   [-0.818147, -0.719535, -1.306853, -2.208709]
cdf(x):      [0.117503, 0.675348, 0.864665, 0.956063]
logcdf(x):   [-2.141291, -0.392528, -0.145413, -0.044931]
cgf(t):      [-0.300370, 0.000000, 0.327210]
cumulants:   [1.25331, 0.429204, 0.17746, 0.0451493]
sample({4}): [1.293910, 0.517537, 1.220869, 1.247025]
sample_like: [1.293910, 0.517537, 1.220869, 1.247025] dtype=11
```

3.16 NormalTrunc

3.16.1 Signature

```
pluss::distr::NormalTrunc normal_trunc{lower, upper, mean, stddev};

normal_trunc.sample(sizes, cfg = {}, seed = 0);
normal_trunc.sample_like(like, seed = 0);
normal_trunc.pdf(x);
normal_trunc.logpdf(x);
normal_trunc.cdf(x);
normal_trunc.logcdf(x);
normal_trunc.cgf(t);
normal_trunc.cumulant(order);
```

3.16.2 Notes

Represents a Gaussian truncated to `[lower, upper]`. The support bounds change both the normalization constant and the low-order cumulants, so it is useful when hard limits are part of the model.

3.16.3 Examples

```
pluss::distr::NormalTrunc normal_trunc{-1.0, 1.0, 0.0, 1.0};
auto x = pluss::make_series({-1.0, 0.0, 0.5, 1.0});
auto like = pluss::zeros({4},
    pluss::Config{}.with_dtype(pluss::ScalarType::Float32).with_platform(pluss::pCPU));
auto t = pluss::make_series({-0.25, 0.0, 0.25});

auto dens = normal_trunc.pdf(x);
auto logdens = normal_trunc.logpdf(x);
auto dist = normal_trunc.cdf(x);
auto logtail = normal_trunc.logcdf(x);
auto cgf = normal_trunc.cgf(t);
auto k1 = normal_trunc.cumulant(1);
auto k2 = normal_trunc.cumulant(2);
auto draws = normal_trunc.sample({4}, pluss::Config{}.with_
↪dtype(pluss::ScalarType::Float64), 20260303ULL);
auto like_draws = normal_trunc.sample_like(like, 20260303ULL);
```

Output

```
pdf(x):      [0.354437, 0.584369, 0.515703, 0.354437]
logpdf(x):   [-1.037223, -0.537223, -0.662223, -1.037223]
cdf(x):      [0.000000, 0.500000, 0.780453, 1.000000]
logcdf(x):   [-inf, -0.693147, -0.247880, 0.000000]
cgf(t):      [0.009083, 0.000000, 0.009083]
cumulants:   [0, 0.291125, 0, -0.0897611]
sample({4}): [-0.114969, 0.692866, -0.043461, -0.069298]
sample_like: [-0.114969, 0.692866, -0.043461, -0.069298] dtype=11
```

3.17 StudentT

3.17.1 Signature

```
pluss::distr::StudentT student_t{degrees_of_freedom, location, scale};

student_t.sample(sizes, cfg = {}, seed = 0);
student_t.sample_like(like, seed = 0);
student_t.pdf(x);
student_t.logpdf(x);
student_t.cdf(x);
student_t.logcdf(x);
student_t.cgf(t);
student_t.cumulant(order);
```

3.17.2 Notes

Represents a Student-t law with location and scale. In this implementation the CGF is only defined at `t = 0`, so use `pdf`, `cdf`, and `cumulant(...)` rather than expecting a full moment-generating surface.

3.17.3 Examples

```
pluss::distr::StudentT student_t{9.0, 0.0, 1.0};
auto x = pluss::make_series({-1.0, 0.0, 0.5, 1.0});
auto like = pluss::zeros({4},
    pluss::Config{}.with_dtype(pluss::ScalarType::Float32).with_platform(pluss::pCPU));
auto t = pluss::make_series({0.0});

auto dens = student_t.pdf(x);
auto logdens = student_t.logpdf(x);
auto dist = student_t.cdf(x);
auto logtail = student_t.logcdf(x);
auto cgf = student_t.cgf(t);
auto k1 = student_t.cumulant(1);
auto k2 = student_t.cumulant(2);
auto draws = student_t.sample({4}, pluss::Config{}.with_
↪dtype(pluss::ScalarType::Float64), 20260303ULL);
auto like_draws = student_t.sample_like(like, 20260303ULL);
```

Output

```
pdf(x):      [0.229131, 0.388035, 0.338357, 0.229131]
logpdf(x):   [-1.473463, -0.946660, -1.083655, -1.473463]
cdf(x):      [0.171718, 0.500000, 0.685464, 0.828282]
logcdf(x):   [-1.761901, -0.693147, -0.377659, -0.188402]
cgf(t):      [0.000000]
cumulants:   [0, 1.28571, 0, 1.98367]
sample({4}): [0.127284, 1.882429, -0.485678, -0.595184]
sample_like: [0.127284, 1.882429, -0.485678, -0.595184] dtype=11
```

3.18 FDistribution

3.18.1 Signature

```
pluss::distr::FDistribution f_dist{numerator_degrees_of_freedom, denominator_degrees_of_
↪freedom};

f_dist.sample(sizes, cfg = {}, seed = 0);
f_dist.sample_like(like, seed = 0);
f_dist.pdf(x);
f_dist.logpdf(x);
f_dist.cdf(x);
f_dist.logcdf(x);
f_dist.cgf(t);
f_dist.cumulant(order);
```

3.18.2 Notes

Models the ratio of two scaled chi-square variates. Like `StudentT`, the CGF is only defined at zero in this implementation, so most user code should rely on the density, CDF, and cumulants.

3.18.3 Examples

```
pluss::distr::FDistribution f_dist{5.0, 12.0};
auto x = pluss::make_series({0.5, 1.5, 2.0, 2.5});
auto like = pluss::zeros({4},
    pluss::Config{}.with_dtype(pluss::ScalarType::Float32).with_platform(pluss::pCPU));
auto t = pluss::make_series({0.0});

auto dens = f_dist.pdf(x);
auto logdens = f_dist.logpdf(x);
auto dist = f_dist.cdf(x);
auto logtail = f_dist.logcdf(x);
auto cgf = f_dist.cgf(t);
auto k1 = f_dist.cumulant(1);
auto k2 = f_dist.cumulant(2);
auto draws = f_dist.sample({4}, pluss::Config{}.with_dtype(pluss::ScalarType::Float64),␣
↪20260303ULL);
auto like_draws = f_dist.sample_like(like, 20260303ULL);
```

Output

```
pdf(x):      [0.697776, 0.292235, 0.161377, 0.090342]
logpdf(x):   [-0.359857, -1.230198, -1.824013, -2.404158]
cdf(x):      [0.229236, 0.738883, 0.849074, 0.910176]
logcdf(x):   [-1.473003, -0.302615, -0.163609, -0.094117]
cgf(t):      [0.000000]
cumulants:   [1.2, 1.08, 3.456, 28.3824]
sample({4}): [0.901638, 1.032027, 0.245787, 0.217795]
sample_like: [0.901639, 1.032027, 0.245787, 0.217795] dtype=11
```

3.19 Weibull

3.19.1 Signature

```
pluss::distr::Weibull weibull{shape, scale};

weibull.sample(sizes, cfg = {}, seed = 0);
weibull.sample_like(like, seed = 0);
weibull.pdf(x);
weibull.logpdf(x);
weibull.cdf(x);
weibull.logcdf(x);
weibull.cgf(t);
weibull.cumulant(order);
```

3.19.2 Notes

Encodes the Weibull family directly as an object. It is the natural choice for lifetime and reliability models where shape controls whether hazard is increasing or decreasing.

3.19.3 Examples

```
pluss::distr::Weibull weibull{2.0, 1.5};
auto x = pluss::make_series({0.5, 1.5, 2.0, 2.5});
auto like = pluss::zeros({4},
    pluss::Config{}.with_dtype(pluss::ScalarType::Float32).with_platform(pluss::pCPU));
auto t = pluss::make_series({-0.25, 0.0, 0.25});

auto dens = weibull.pdf(x);
auto logdens = weibull.logpdf(x);
auto dist = weibull.cdf(x);
auto logtail = weibull.logcdf(x);
auto cgf = weibull.cgf(t);
auto k1 = weibull.cumulant(1);
auto k2 = weibull.cumulant(2);
auto draws = weibull.sample({4}, pluss::Config{}.with_dtype(pluss::ScalarType::Float64),␣
↪20260303ULL);
auto like_draws = weibull.sample_like(like, 20260303ULL);
```

Output

```
pdf(x):      [0.397706, 0.490506, 0.300468, 0.138170]
logpdf(x):   [-0.922041, -0.712318, -1.202414, -1.979270]
cdf(x):      [0.105161, 0.632121, 0.830987, 0.937823]
logcdf(x):   [-2.252266, -0.458675, -0.185142, -0.064194]
cgf(t):      [-0.317788, 0.000000, 0.347985]
cumulants:   [1.32934, 0.482854, 0.211753, 0.0571421]
sample({4}): [1.372398, 0.548931, 1.294927, 1.322670]
sample_like: [1.372398, 0.548931, 1.294927, 1.322670] dtype=11
```

3.20 Beta

3.20.1 Signature

```
pluss::distr::Beta beta{low, high, alpha, beta};

beta.sample(sizes, cfg = {}, seed = 0);
beta.sample_like(like, seed = 0);
beta.pdf(x);
beta.logpdf(x);
beta.cdf(x);
beta.logcdf(x);
beta.cgf(t);
beta.cumulant(order);
```

3.20.2 Notes

Represents a beta law rescaled onto `[low, high]`. Use it for bounded proportions or rates when you want both support constraints and adjustable skew.

3.20.3 Examples

```
pluss::distr::Beta beta{0.0, 1.0, 2.0, 3.0};
auto x = pluss::make_series({0.0, 0.4, 0.6, 0.8});
auto like = pluss::zeros({4},
    pluss::Config{}.with_dtype(pluss::ScalarType::Float32).with_platform(pluss::pCPU));
auto t = pluss::make_series({-0.25, 0.0, 0.25});

auto dens = beta.pdf(x);
auto logdens = beta.logpdf(x);
auto dist = beta.cdf(x);
auto logtail = beta.logcdf(x);
auto cgf = beta.cgf(t);
auto k1 = beta.cumulant(1);
auto k2 = beta.cumulant(2);
auto draws = beta.sample({4}, pluss::Config{}.with_dtype(pluss::ScalarType::Float64),␣
↪20260303ULL);
auto like_draws = beta.sample_like(like, 20260303ULL);
```

Output

```
pdf(x):      [0.000000, 1.728000, 1.152000, 0.384000]
logpdf(x):   [-inf, 0.546965, 0.141500, -0.957113]
cdf(x):      [0.000000, 0.524800, 0.820800, 0.972800]
logcdf(x):   [-inf, -0.644738, -0.197476, -0.027577]
cgf(t):      [-0.098756, 0.000000, 0.101256]
cumulants:   [0.4, 0.04, 0.00228571, -0.00102857]
sample({4}): [0.375967, 0.417967, 0.096186, 0.169165]
sample_like: [0.375967, 0.417967, 0.096186, 0.169165] dtype=11
```

3.21 Skellam

3.21.1 Signature

```
pluss::distr::Skellam skellam{lambda1, lambda2};

skellam.sample(sizes, cfg = {}, seed = 0);
skellam.sample_like(like, seed = 0);
skellam.pdf(x);
skellam.logpdf(x);
skellam.cdf(x);
skellam.logcdf(x);
skellam.cgf(t);
skellam.cumulant(order);
```

3.21.2 Notes

Represents the difference of two independent Poisson counts. Odd cumulants track the rate imbalance and even cumulants track total count variability.

3.21.3 Examples

```
pluss::distr::Skellam skellam{2.0, 1.0};
auto x = pluss::make_series({-1.0, 0.0, 1.0, 2.0});
auto like = pluss::zeros({4},
    pluss::Config{}.with_dtype(pluss::ScalarType::Float32).with_platform(pluss::pCPU));
auto t = pluss::make_series({-0.25, 0.0, 0.25});

auto pmf = skellam.pdf(x);
auto logprob = skellam.logpdf(x);
auto dist = skellam.cdf(x);
auto logtail = skellam.logcdf(x);
auto cgf = skellam.cgf(t);
auto k1 = skellam.cumulant(1);
auto k2 = skellam.cumulant(2);
auto draws = skellam.sample({4}, pluss::Config{}.with_dtype(pluss::ScalarType::Float64),␣
↪20260303ULL);
auto like_draws = skellam.sample_like(like, 20260303ULL);
```

Output

```
pdf(x):      [0.119232, 0.211712, 0.238463, 0.184961]
logpdf(x):   [-2.126686, -1.552528, -1.433539, -1.687612]
cdf(x):      [0.182585, 0.394297, 0.632760, 0.817721]
logcdf(x):   [-1.700541, -0.930651, -0.457664, -0.201234]
cgf(t):      [-0.158373, 0.000000, 0.346852]
cumulants:   [1, 3, 1, 3]
sample({4}): [5.000000, 3.000000, 4.000000, 0.000000]
sample_like: [5.000000, 3.000000, 4.000000, 0.000000] dtype=11
```

3.22 Nig

3.22.1 Signature

```
pluss::distr::Nig nig{alpha, beta, location, scale};

nig.sample(sizes, cfg = {}, seed = 0);
nig.sample_like(like, seed = 0);
nig.pdf(x);
nig.logpdf(x);
nig.cdf(x);
nig.logcdf(x);
nig.cgf(t);
nig.cumulant(order);
```

3.22.2 Notes

Normal-inverse-Gaussian wrapper for skewed heavy-tailed location-scale modeling. `cumulant(order)` currently supports exact closed forms up to order 4.

3.22.3 Examples

```
pluss::distr::Nig nig{2.0, 0.5, 0.0, 1.0};
auto x = pluss::make_series({-1.0, 0.0, 0.5, 1.0});
auto like = pluss::zeros({4},
    pluss::Config{}.with_dtype(pluss::ScalarType::Float32).with_platform(pluss::pCPU));
auto t = pluss::make_series({-0.25, 0.0, 0.25});

auto dens = nig.pdf(x);
auto logdens = nig.logpdf(x);
auto dist = nig.cdf(x);
auto logtail = nig.logcdf(x);
auto cgf = nig.cgf(t);
auto k1 = nig.cumulant(1);
auto k2 = nig.cumulant(2);
auto draws = nig.sample({4}, pluss::Config{}.with_dtype(pluss::ScalarType::Float64),␣
↪20260303ULL);
auto like_draws = nig.sample_like(like, 20260303ULL);
```

Output

```
pdf(x):      [0.093492, 0.617447, 0.522367, 0.254138]
logpdf(x):   [-2.369877, -0.482163, -0.649385, -1.369877]
cdf(x):      [0.033036, 0.367565, 0.671088, 0.862480]
logcdf(x):   [-3.410163, -1.000856, -0.398855, -0.147944]
cgf(t):      [-0.047822, 0.000000, 0.082442]
cumulants:   [0.258199, 0.550824, 0.22033, 0.587546]
sample({4}): [0.276759, 1.896829, 1.379230, -0.724068]
sample_like: [0.276759, 1.896829, 1.379230, -0.724068] dtype=11
```

3.23 Gh

3.23.1 Signature

```
pluss::distr::Gh gh{lambda, alpha, beta, location, scale};

gh.sample(sizes, cfg = {}, seed = 0);
gh.sample_like(like, seed = 0);
gh.pdf(x);
gh.logpdf(x);
gh.cdf(x);
gh.logcdf(x);
gh.cgf(t);
gh.cumulant(order);
```

3.23.2 Notes

Generalized hyperbolic wrapper for asymmetric heavy-tailed modeling. It generalizes `Nig` by exposing the extra shape parameter `lambda` while retaining the same object-oriented method surface.

3.23.3 Examples

```
pluss::distr::Gh gh{1.0, 2.0, 0.5, 0.0, 1.0};
auto x = pluss::make_series({-1.0, 0.0, 0.5, 1.0});
auto like = pluss::zeros({4},
    pluss::Config{}.with_dtype(pluss::ScalarType::Float32).with_platform(pluss::pCPU));
auto t = pluss::make_series({-0.25, 0.0, 0.25});

auto dens = gh.pdf(x);
auto logdens = gh.logpdf(x);
auto dist = gh.cdf(x);
auto logtail = gh.logcdf(x);
auto cgf = gh.cgf(t);
auto k1 = gh.cumulant(1);
auto k2 = gh.cumulant(2);
auto draws = gh.sample({4}, pluss::Config{}.with_dtype(pluss::ScalarType::Float64),␣
↪20260303ULL);
auto like_draws = gh.sample_like(like, 20260303ULL);
```

Output

```
pdf(x):      [0.114108, 0.430768, 0.436811, 0.310177]
logpdf(x):   [-2.170613, -0.842186, -0.828254, -1.170613]
cdf(x):      [0.054111, 0.322720, 0.547947, 0.736934]
logcdf(x):   [-2.916726, -1.130971, -0.601576, -0.305257]
cgf(t):      [-0.087597, 0.000000, 0.153511]
cumulants:   [0.475739, 1.04555, 0.610504, 1.71741]
sample({4}): [2.037122, -0.171804, -0.721009, 2.011074]
sample_like: [2.037122, -0.171804, -0.721009, 2.011074] dtype=11
```

3.24 GaussianMixture

3.24.1 Signature

```
pluss::distr::GaussianMixture gm{weights, means, stddevs};

gm.sample(sizes, cfg = {}, seed = 0, use_sobol = false);
gm.sample_like(like, seed = 0, use_sobol = false);
gm.pdf(x);
gm.logpdf(x);
gm.cdf(x);
gm.logcdf(x);
gm.cgf(t);
gm.cumulant(order);
```

3.24.2 Notes

Wraps a Gaussian mixture with component weights, means, and standard deviations stored as `Series`. The wrapper validates component-shape compatibility and supports `use_sobol=true` for Sobol-driven sampling.

3.24.3 Examples

```
auto weights = pluss::make_series({0.7, 0.3});
auto means = pluss::make_series({-1.0, 2.0});
auto stddevs = pluss::make_series({0.5, 1.25});
pluss::distr::GaussianMixture gm{weights, means, stddevs};
auto x = pluss::make_series({-1.0, 0.0, 0.5, 1.0});
auto like = pluss::zeros({4},
    pluss::Config{}.with_dtype(pluss::ScalarType::Float32).with_platform(pluss::pCPU));
auto t = pluss::make_series({-0.25, 0.0, 0.25});

auto dens = gm.pdf(x);
auto logdens = gm.logpdf(x);
auto dist = gm.cdf(x);
auto logtail = gm.logcdf(x);
auto cgf = gm.cgf(t);
auto k1 = gm.cumulant(1);
auto k2 = gm.cumulant(2);
auto draws = gm.sample({4}, pluss::Config{}.with_dtype(pluss::ScalarType::Float64),␣
↪20260303ULL);
auto like_draws = gm.sample_like(like, 20260303ULL);
```

Output

```
pdf(x):      [0.563894, 0.102208, 0.052809, 0.069713]
logpdf(x):   [-0.572889, -2.280742, -2.941069, -2.663364]
cdf(x):      [0.352459, 0.700515, 0.733576, 0.763534]
logcdf(x):   [-1.042820, -0.355940, -0.309824, -0.269797]
cgf(t):      [0.092517, 0.000000, 0.066540]
cumulants:   [-0.1, 2.53375, 4.74862, 2.61617]
sample({4}): [0.853880, 2.383822, -0.691794, 1.097873]
sample_like: [0.853880, 2.383822, -0.691794, 1.097873] dtype=11
```

CHAPTER

FOUR

DISTRIBUTION CATALOG

Each section below documents one supported family with the same reference layout used throughout the docs: `Signature`, `Notes`, and `Examples`.

4.1 normal

4.1.1 Signature

```
pluss::distr::sample("normal", {mu, sigma}, sizes, cfg, seed);
pluss::distr::sample("normal", {}, sizes, cfg, seed);
pluss::distr::sample_like("normal", {mu, sigma}, like, seed);
pluss::distr::sample_like("normal", {}, like, seed);
pluss::distr::pdf("normal", x, {mu, sigma});
pluss::distr::pdf("normal", x, {});
pluss::distr::logpdf("normal", x, {mu, sigma});
pluss::distr::logpdf("normal", x, {});
pluss::distr::cdf("normal", x, {mu, sigma});
pluss::distr::cdf("normal", x, {});
```

4.1.2 Notes

Gaussian family. Use the empty parameter list for the standard normal, or pass mean and standard deviation for a shifted/scaled normal.

Aliases: `norm`, `gaussian`, `gauss`.

Parameters: `{mu, sigma}` or `{}` for standard normal.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

4.1.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({-1.0, 0.0, 1.0}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("normal", {0.0, 1.0}, {6}, cfg, 11ULL);
auto like_draws = pluss::distr::sample_like("normal", {0.0, 1.0}, like, 11ULL);
```

(continues on next page)

(continued from previous page)

```
auto dens = pluss::distr::pdf("normal", x, {0.0, 1.0});
auto logprob = pluss::distr::logpdf("normal", x, {0.0, 1.0});
auto dist = pluss::distr::cdf("normal", x, {0.0, 1.0});
```

4.2 lognormal

4.2.1 Signature

```
pluss::distr::sample("lognormal", {mu, sigma}, sizes, cfg, seed);
pluss::distr::sample_like("lognormal", {mu, sigma}, like, seed);
pluss::distr::pdf("lognormal", x, {mu, sigma});
pluss::distr::logpdf("lognormal", x, {mu, sigma});
pluss::distr::cdf("lognormal", x, {mu, sigma});
```

4.2.2 Notes

Samples and densities are computed in log-space, then mapped back to the positive real line.

Aliases: `lognorm, cobbdouglas, antilognormal`.

Parameters: `{mu, sigma}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

4.2.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({0.5, 1.0, 2.0}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("lognorm", {0.0, 0.5}, {6}, cfg, 12ULL);
auto like_draws = pluss::distr::sample_like("lognorm", {0.0, 0.5}, like, 12ULL);
auto dens = pluss::distr::pdf("lognorm", x, {0.0, 0.5});
auto logprob = pluss::distr::logpdf("lognorm", x, {0.0, 0.5});
auto dist = pluss::distr::cdf("lognorm", x, {0.0, 0.5});
```

4.3 uniform

4.3.1 Signature

```
pluss::distr::sample("uniform", {a, b}, sizes, cfg, seed);
pluss::distr::sample_like("uniform", {a, b}, like, seed);
pluss::distr::pdf("uniform", x, {a, b});
pluss::distr::logpdf("uniform", x, {a, b});
pluss::distr::cdf("uniform", x, {a, b});
```

4.3.2 Notes

Draws uniformly on the closed interval defined by the lower and upper bounds.

Aliases: `unif`.

Parameters: `{a, b}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

4.3.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({0.0, 0.25, 0.5, 0.75, 1.0}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("uniform", {0.0, 1.0}, {6}, cfg, 13ULL);
auto like_draws = pluss::distr::sample_like("uniform", {0.0, 1.0}, like, 13ULL);
auto dens = pluss::distr::pdf("uniform", x, {0.0, 1.0});
auto logprob = pluss::distr::logpdf("uniform", x, {0.0, 1.0});
auto dist = pluss::distr::cdf("uniform", x, {0.0, 1.0});
```

4.4 exponential

4.4.1 Signature

```
pluss::distr::sample("exponential", {lambda}, sizes, cfg, seed);
pluss::distr::sample_like("exponential", {lambda}, like, seed);
pluss::distr::pdf("exponential", x, {lambda});
pluss::distr::logpdf("exponential", x, {lambda});
pluss::distr::cdf("exponential", x, {lambda});
```

4.4.2 Notes

Uses the rate parameter lambda. Larger lambda concentrates more mass near zero.

Aliases: `exp`.

Parameters: `{lambda}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

4.4.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({0.0, 0.5, 1.0, 1.5}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("exp", {1.5}, {6}, cfg, 14ULL);
```

(continues on next page)

(continued from previous page)

```
auto like_draws = pluss::distr::sample_like("exp", {1.5}, like, 14ULL);
auto dens = pluss::distr::pdf("exp", x, {1.5});
auto logprob = pluss::distr::logpdf("exp", x, {1.5});
auto dist = pluss::distr::cdf("exp", x, {1.5});
```

4.5 bernoulli

4.5.1 Signature

```
pluss::distr::sample("bernoulli", {p}, sizes, cfg, seed);
pluss::distr::sample_like("bernoulli", {p}, like, seed);
pluss::distr::pdf("bernoulli", x, {p});
pluss::distr::logpdf("bernoulli", x, {p});
pluss::distr::cdf("bernoulli", x, {p});
```

4.5.2 Notes

Discrete 0/1 family. `sample(...)` naturally produces a boolean Series unless you override dtype.

Aliases: `bern`.

Parameters: `{p}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

4.5.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({0.0, 1.0}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("bern", {0.35}, {8}, cfg, 15ULL);
auto like_draws = pluss::distr::sample_like("bern", {0.35}, like, 15ULL);
auto pmf = pluss::distr::pdf("bern", x, {0.35});
auto logprob = pluss::distr::logpdf("bern", x, {0.35});
auto dist = pluss::distr::cdf("bern", x, {0.35});
```

4.6 bessel

4.6.1 Signature

```
pluss::distr::sample("bessel", {nu, a}, sizes, cfg, seed);
pluss::distr::sample_like("bessel", {nu, a}, like, seed);
pluss::distr::pdf("bessel", x, {nu, a});
pluss::distr::logpdf("bessel", x, {nu, a});
pluss::distr::cdf("bessel", x, {nu, a});
```

4.6.2 Notes

Discrete non-negative count family normalized by the modified Bessel function `I_nu(a)`. `nu` must stay above `-1` and `a` must be positive. This is the right model when the count law itself is defined through a Bessel-function normalization rather than through a Poisson or negative-binomial construction.

Aliases: none.

Parameters: `{nu, a}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

4.6.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({0.0, 1.0, 2.0, 3.0}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("bessel", {0.0, 2.5}, {6}, cfg, 15ULL);
auto like_draws = pluss::distr::sample_like("bessel", {0.0, 2.5}, like, 15ULL);
auto pmf = pluss::distr::pdf("bessel", x, {0.0, 2.5});
auto logprob = pluss::distr::logpdf("bessel", x, {0.0, 2.5});
auto dist = pluss::distr::cdf("bessel", x, {0.0, 2.5});
```

4.7 beta

4.7.1 Signature

```
pluss::distr::sample("beta", {a, b}, sizes, cfg, seed);
pluss::distr::sample("beta", {m, n, a, b}, sizes, cfg, seed);
pluss::distr::sample_like("beta", {a, b}, like, seed);
pluss::distr::sample_like("beta", {m, n, a, b}, like, seed);
pluss::distr::pdf("beta", x, {a, b});
pluss::distr::pdf("beta", x, {m, n, a, b});
pluss::distr::logpdf("beta", x, {a, b});
pluss::distr::logpdf("beta", x, {m, n, a, b});
pluss::distr::cdf("beta", x, {a, b});
pluss::distr::cdf("beta", x, {m, n, a, b});
```

4.7.2 Notes

Use the two-parameter form for the standard beta on [0,1]. The four-parameter form exposes the shifted/scaled legacy convention.

Aliases: `powerfunction, powerfunc`.

Parameters: `{a, b} or {m, n, a, b}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

4.7.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({0.25, 0.5, 0.75}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("beta", {2.0, 5.0}, {6}, cfg, 16ULL);
auto like_draws = pluss::distr::sample_like("beta", {2.0, 5.0}, like, 16ULL);
auto dens = pluss::distr::pdf("beta", x, {2.0, 5.0});
auto logprob = pluss::distr::logpdf("beta", x, {2.0, 5.0});
auto dist = pluss::distr::cdf("beta", x, {2.0, 5.0});
```

4.8 binomial

4.8.1 Signature

```
pluss::distr::sample("binomial", {n, p}, sizes, cfg, seed);
pluss::distr::sample_like("binomial", {n, p}, like, seed);
pluss::distr::pdf("binomial", x, {n, p});
pluss::distr::logpdf("binomial", x, {n, p});
pluss::distr::cdf("binomial", x, {n, p});
```

4.8.2 Notes

Counts successes across `n` Bernoulli trials.

Aliases: `binom`.

Parameters: `{n, p}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

4.8.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({0.0, 1.0, 2.0, 3.0}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("binom", {10.0, 0.3}, {6}, cfg, 17ULL);
auto like_draws = pluss::distr::sample_like("binom", {10.0, 0.3}, like, 17ULL);
auto pmf = pluss::distr::pdf("binom", x, {10.0, 0.3});
auto logprob = pluss::distr::logpdf("binom", x, {10.0, 0.3});
auto dist = pluss::distr::cdf("binom", x, {10.0, 0.3});
```

4.9 bradford

4.9.1 Signature

```
pluss::distr::sample("bradford", {c}, sizes, cfg, seed);
pluss::distr::sample_like("bradford", {c}, like, seed);
pluss::distr::pdf("bradford", x, {c});
pluss::distr::logpdf("bradford", x, {c});
pluss::distr::cdf("bradford", x, {c});
```

4.9.2 Notes

Bounded skewed family controlled by the shape parameter `c`.

Aliases: none.

Parameters: `{c}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

4.9.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({0.1, 0.4, 0.8}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("bradford", {1.5}, {6}, cfg, 18ULL);
auto like_draws = pluss::distr::sample_like("bradford", {1.5}, like, 18ULL);
auto dens = pluss::distr::pdf("bradford", x, {1.5});
auto logprob = pluss::distr::logpdf("bradford", x, {1.5});
auto dist = pluss::distr::cdf("bradford", x, {1.5});
```

4.10 burr

4.10.1 Signature

```
pluss::distr::sample("burr", {c, k}, sizes, cfg, seed);
pluss::distr::sample_like("burr", {c, k}, like, seed);
pluss::distr::pdf("burr", x, {c, k});
pluss::distr::logpdf("burr", x, {c, k});
pluss::distr::cdf("burr", x, {c, k});
```

4.10.2 Notes

Heavy-tailed positive family. Both parameters control tail shape.

Aliases: `fisk`.

Parameters: `{c, k}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

4.10.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({1.0, 2.0, 3.0}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("burr", {2.0, 3.0}, {6}, cfg, 19ULL);
auto like_draws = pluss::distr::sample_like("burr", {2.0, 3.0}, like, 19ULL);
auto dens = pluss::distr::pdf("burr", x, {2.0, 3.0});
auto logprob = pluss::distr::logpdf("burr", x, {2.0, 3.0});
auto dist = pluss::distr::cdf("burr", x, {2.0, 3.0});
```

4.11 cauchy

4.11.1 Signature

```
pluss::distr::sample("cauchy", {location, scale}, sizes, cfg, seed);
pluss::distr::sample_like("cauchy", {location, scale}, like, seed);
pluss::distr::pdf("cauchy", x, {location, scale});
pluss::distr::logpdf("cauchy", x, {location, scale});
pluss::distr::cdf("cauchy", x, {location, scale});
```

4.11.2 Notes

Heavy-tailed location-scale family with undefined mean/variance in the classical sense.

Aliases: `lorentz, caushy`.

Parameters: `{location, scale}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

4.11.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({-1.0, 0.0, 1.0}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("cauchy", {0.0, 1.0}, {6}, cfg, 20ULL);
auto like_draws = pluss::distr::sample_like("cauchy", {0.0, 1.0}, like, 20ULL);
auto dens = pluss::distr::pdf("cauchy", x, {0.0, 1.0});
auto logprob = pluss::distr::logpdf("cauchy", x, {0.0, 1.0});
auto dist = pluss::distr::cdf("cauchy", x, {0.0, 1.0});
```

4.12 chi

4.12.1 Signature

```
pluss::distr::sample("chi", {nu}, sizes, cfg, seed);
pluss::distr::sample("chi", {a, b, nu}, sizes, cfg, seed);
pluss::distr::sample_like("chi", {nu}, like, seed);
pluss::distr::sample_like("chi", {a, b, nu}, like, seed);
pluss::distr::pdf("chi", x, {nu});
pluss::distr::pdf("chi", x, {a, b, nu});
pluss::distr::logpdf("chi", x, {nu});
pluss::distr::logpdf("chi", x, {a, b, nu});
pluss::distr::cdf("chi", x, {nu});
pluss::distr::cdf("chi", x, {a, b, nu});
```

4.12.2 Notes

Chi family. The short form is the standard chi distribution; the longer legacy form exposes location/scale/shape.

Aliases: none.

Parameters: `{nu} or {a, b, nu}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

4.12.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({0.5, 1.0, 2.0}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("chi", {3.0}, {6}, cfg, 21ULL);
auto like_draws = pluss::distr::sample_like("chi", {3.0}, like, 21ULL);
auto dens = pluss::distr::pdf("chi", x, {3.0});
auto logprob = pluss::distr::logpdf("chi", x, {3.0});
auto dist = pluss::distr::cdf("chi", x, {3.0});
```

4.13 chisquare

4.13.1 Signature

```
pluss::distr::sample("chisquare", {nu}, sizes, cfg, seed);
pluss::distr::sample_like("chisquare", {nu}, like, seed);
pluss::distr::pdf("chisquare", x, {nu});
pluss::distr::logpdf("chisquare", x, {nu});
pluss::distr::cdf("chisquare", x, {nu});
```

4.13.2 Notes

Standard chi-square family parameterized by degrees of freedom.

Aliases: `chisq, chi2`.

Parameters: `{nu}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

4.13.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({0.5, 1.0, 2.0, 4.0}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("chisq", {4.0}, {6}, cfg, 22ULL);
auto like_draws = pluss::distr::sample_like("chisq", {4.0}, like, 22ULL);
auto dens = pluss::distr::pdf("chisq", x, {4.0});
auto logprob = pluss::distr::logpdf("chisq", x, {4.0});
auto dist = pluss::distr::cdf("chisq", x, {4.0});
```

4.14 chisqnoncentral

4.14.1 Signature

```
pluss::distr::sample("chisqnoncentral", {lambda, nu}, sizes, cfg, seed);
pluss::distr::sample_like("chisqnoncentral", {lambda, nu}, like, seed);
pluss::distr::pdf("chisqnoncentral", x, {lambda, nu});
pluss::distr::logpdf("chisqnoncentral", x, {lambda, nu});
pluss::distr::cdf("chisqnoncentral", x, {lambda, nu});
```

4.14.2 Notes

Non-central chi-square family with non-centrality `lambda` and degrees of freedom `nu`.

Aliases: `chisqnc, chisqnoncentral, chisqnoncentr, chi2noncentral`.

Parameters: `{lambda, nu}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

4.14.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({1.0, 2.0, 3.0}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("chisqnc", {1.5, 4.0}, {6}, cfg, 23ULL);
```

(continues on next page)

(continued from previous page)

```
auto like_draws = pluss::distr::sample_like("chisqnc", {1.5, 4.0}, like, 23ULL);
auto dens = pluss::distr::pdf("chisqnc", x, {1.5, 4.0});
auto logprob = pluss::distr::logpdf("chisqnc", x, {1.5, 4.0});
auto dist = pluss::distr::cdf("chisqnc", x, {1.5, 4.0});
```

4.15 cosine

4.15.1 Signature

```
pluss::distr::sample("cosine", {}, sizes, cfg, seed);
pluss::distr::sample_like("cosine", {}, like, seed);
pluss::distr::pdf("cosine", x, {});
pluss::distr::logpdf("cosine", x, {});
pluss::distr::cdf("cosine", x, {});
```

4.15.2 Notes

Compact-support cosine density centered at zero.

Aliases: none.

Parameters: `{}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

4.15.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({-1.0, 0.0, 1.0}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("cosine", {}, {6}, cfg, 24ULL);
auto like_draws = pluss::distr::sample_like("cosine", like, 24ULL);
auto dens = pluss::distr::pdf("cosine", x, {});
auto logprob = pluss::distr::logpdf("cosine", x, {});
auto dist = pluss::distr::cdf("cosine", x, {});
```

4.16 erlang

4.16.1 Signature

```
pluss::distr::sample("erlang", {scale, k}, sizes, cfg, seed);
pluss::distr::sample_like("erlang", {scale, k}, like, seed);
pluss::distr::pdf("erlang", x, {scale, k});
pluss::distr::logpdf("erlang", x, {scale, k});
pluss::distr::cdf("erlang", x, {scale, k});
```

4.16.2 Notes

Integer-shape gamma special case. Useful when you want the classic Erlang parameterization.

Aliases: none.

Parameters: `{scale, k}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

4.16.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({0.5, 1.0, 2.0}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("erlang", {1.25, 3.0}, {6}, cfg, 25ULL);
auto like_draws = pluss::distr::sample_like("erlang", {1.25, 3.0}, like, 25ULL);
auto dens = pluss::distr::pdf("erlang", x, {1.25, 3.0});
auto logprob = pluss::distr::logpdf("erlang", x, {1.25, 3.0});
auto dist = pluss::distr::cdf("erlang", x, {1.25, 3.0});
```

4.17 extremevalue

4.17.1 Signature

```
pluss::distr::sample("extremevalue", {location, scale}, sizes, cfg, seed);
pluss::distr::sample_like("extremevalue", {location, scale}, like, seed);
pluss::distr::pdf("extremevalue", x, {location, scale});
pluss::distr::logpdf("extremevalue", x, {location, scale});
pluss::distr::cdf("extremevalue", x, {location, scale});
```

4.17.2 Notes

Gumbel-type extreme-value distribution.

Aliases: `extrval, extrvalue, gumbel`.

Parameters: `{location, scale}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

4.17.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({-1.0, 0.0, 1.0}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("extrval", {0.0, 1.0}, {6}, cfg, 26ULL);
```

(continues on next page)

(continued from previous page)

```
auto like_draws = pluss::distr::sample_like("extrval", {0.0, 1.0}, like, 26ULL);
auto dens = pluss::distr::pdf("extrval", x, {0.0, 1.0});
auto logprob = pluss::distr::logpdf("extrval", x, {0.0, 1.0});
auto dist = pluss::distr::cdf("extrval", x, {0.0, 1.0});
```

4.18 f

4.18.1 Signature

```
pluss::distr::sample("f", {df1, df2}, sizes, cfg, seed);
pluss::distr::sample_like("f", {df1, df2}, like, seed);
pluss::distr::pdf("f", x, {df1, df2});
pluss::distr::logpdf("f", x, {df1, df2});
pluss::distr::cdf("f", x, {df1, df2});
```

4.18.2 Notes

Classical F family with numerator and denominator degrees of freedom.

Aliases: `fdistribution`, `fdistrib`, `fdistr`, `fdist`, `fdis`.

Parameters: `{df1, df2}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

4.18.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({0.5, 1.0, 2.0}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("f", {4.0, 6.0}, {6}, cfg, 27ULL);
auto like_draws = pluss::distr::sample_like("f", {4.0, 6.0}, like, 27ULL);
auto dens = pluss::distr::pdf("f", x, {4.0, 6.0});
auto logprob = pluss::distr::logpdf("f", x, {4.0, 6.0});
auto dist = pluss::distr::cdf("f", x, {4.0, 6.0});
```

4.19 fnoncentral

4.19.1 Signature

```
pluss::distr::sample("fnoncentral", {lambda, df1, df2}, sizes, cfg, seed);
pluss::distr::sample_like("fnoncentral", {lambda, df1, df2}, like, seed);
pluss::distr::pdf("fnoncentral", x, {lambda, df1, df2});
pluss::distr::logpdf("fnoncentral", x, {lambda, df1, df2});
pluss::distr::cdf("fnoncentral", x, {lambda, df1, df2});
```

4.19.2 Notes

Non-central F family with non-centrality and two degree-of-freedom parameters.

Aliases: `fnc, fnoncentral, fnoncentr`.

Parameters: `{lambda, df1, df2}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

4.19.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({0.5, 1.0, 2.0}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("fnc", {1.5, 4.0, 6.0}, {6}, cfg, 28ULL);
auto like_draws = pluss::distr::sample_like("fnc", {1.5, 4.0, 6.0}, like, 28ULL);
auto dens = pluss::distr::pdf("fnc", x, {1.5, 4.0, 6.0});
auto logprob = pluss::distr::logpdf("fnc", x, {1.5, 4.0, 6.0});
auto dist = pluss::distr::cdf("fnc", x, {1.5, 4.0, 6.0});
```

4.20 laplace

4.20.1 Signature

```
pluss::distr::sample("laplace", {location, scale}, sizes, cfg, seed);
pluss::distr::sample_like("laplace", {location, scale}, like, seed);
pluss::distr::pdf("laplace", x, {location, scale});
pluss::distr::logpdf("laplace", x, {location, scale});
pluss::distr::cdf("laplace", x, {location, scale});
```

4.20.2 Notes

Two-sided exponential family centered at `location`.

Aliases: `doubleexponential, doubleexp, bilateralexponential, bilateralexp`.

Parameters: `{location, scale}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

4.20.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({-1.0, 0.0, 1.0}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("laplace", {0.0, 1.0}, {6}, cfg, 29ULL);
```

(continues on next page)

(continued from previous page)

```
auto like_draws = pluss::distr::sample_like("laplace", {0.0, 1.0}, like, 29ULL);
auto dens = pluss::distr::pdf("laplace", x, {0.0, 1.0});
auto logprob = pluss::distr::logpdf("laplace", x, {0.0, 1.0});
auto dist = pluss::distr::cdf("laplace", x, {0.0, 1.0});
```

4.21 logistic

4.21.1 Signature

```
pluss::distr::sample("logistic", {location, scale}, sizes, cfg, seed);
pluss::distr::sample_like("logistic", {location, scale}, like, seed);
pluss::distr::pdf("logistic", x, {location, scale});
pluss::distr::logpdf("logistic", x, {location, scale});
pluss::distr::cdf("logistic", x, {location, scale});
```

4.21.2 Notes

Logistic location-scale family.

Aliases: none.

Parameters: `{location, scale}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

4.21.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({-1.0, 0.0, 1.0}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("logistic", {0.0, 1.0}, {6}, cfg, 30ULL);
auto like_draws = pluss::distr::sample_like("logistic", {0.0, 1.0}, like, 30ULL);
auto dens = pluss::distr::pdf("logistic", x, {0.0, 1.0});
auto logprob = pluss::distr::logpdf("logistic", x, {0.0, 1.0});
auto dist = pluss::distr::cdf("logistic", x, {0.0, 1.0});
```

4.22 geometric

4.22.1 Signature

```
pluss::distr::sample("geometric", {p}, sizes, cfg, seed);
pluss::distr::sample_like("geometric", {p}, like, seed);
pluss::distr::pdf("geometric", x, {p});
pluss::distr::logpdf("geometric", x, {p});
pluss::distr::cdf("geometric", x, {p});
```

4.22.2 Notes

Counts trials until first success under the runtime convention implemented in `distr.cpp`.

Aliases: `geom, furry`.

Parameters: `{p}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

4.22.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({1.0, 2.0, 3.0}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("geom", {0.35}, {6}, cfg, 31ULL);
auto like_draws = pluss::distr::sample_like("geom", {0.35}, like, 31ULL);
auto dens = pluss::distr::pdf("geom", x, {0.35});
auto logprob = pluss::distr::logpdf("geom", x, {0.35});
auto dist = pluss::distr::cdf("geom", x, {0.35});
```

4.23 gamma

4.23.1 Signature

```
pluss::distr::sample("gamma", {a}, sizes, cfg, seed);
pluss::distr::sample("gamma", {m, b, a}, sizes, cfg, seed);
pluss::distr::sample_like("gamma", {a}, like, seed);
pluss::distr::sample_like("gamma", {m, b, a}, like, seed);
pluss::distr::pdf("gamma", x, {a});
pluss::distr::pdf("gamma", x, {m, b, a});
pluss::distr::logpdf("gamma", x, {a});
pluss::distr::logpdf("gamma", x, {m, b, a});
pluss::distr::cdf("gamma", x, {a});
pluss::distr::cdf("gamma", x, {m, b, a});
```

4.23.2 Notes

Gamma family. Use the one-parameter shortcut for the standard shape-only form, or the legacy location/scale/shape form when needed.

Aliases: none.

Parameters: `{a} or {m, b, a}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

4.23.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({0.5, 1.0, 2.0}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("gamma", {2.5}, {6}, cfg, 32ULL);
auto like_draws = pluss::distr::sample_like("gamma", {2.5}, like, 32ULL);
auto dens = pluss::distr::pdf("gamma", x, {2.5});
auto logprob = pluss::distr::logpdf("gamma", x, {2.5});
auto dist = pluss::distr::cdf("gamma", x, {2.5});
```

4.24 gompertz

4.24.1 Signature

```
pluss::distr::sample("gompertz", {eta, b}, sizes, cfg, seed);
pluss::distr::sample_like("gompertz", {eta, b}, like, seed);
pluss::distr::pdf("gompertz", x, {eta, b});
pluss::distr::logpdf("gompertz", x, {eta, b});
pluss::distr::cdf("gompertz", x, {eta, b});
```

4.24.2 Notes

Positive-support Gompertz family.

Aliases: none.

Parameters: `{eta, b}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

4.24.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({0.25, 0.5, 1.0}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("gompertz", {1.0, 2.0}, {6}, cfg, 33ULL);
auto like_draws = pluss::distr::sample_like("gompertz", {1.0, 2.0}, like, 33ULL);
auto dens = pluss::distr::pdf("gompertz", x, {1.0, 2.0});
auto logprob = pluss::distr::logpdf("gompertz", x, {1.0, 2.0});
auto dist = pluss::distr::cdf("gompertz", x, {1.0, 2.0});
```

4.25 halfcosine

4.25.1 Signature

```
pluss::distr::sample("halfcosine", {}, sizes, cfg, seed);
pluss::distr::sample_like("halfcosine", {}, like, seed);
pluss::distr::pdf("halfcosine", x, {});
pluss::distr::logpdf("halfcosine", x, {});
pluss::distr::cdf("halfcosine", x, {});
```

4.25.2 Notes

Half-cosine density on bounded support.

Aliases: `hcosine, hcos`.

Parameters: `{}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

4.25.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({0.0, 0.25, 0.5}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("hcos", {}, {6}, cfg, 34ULL);
auto like_draws = pluss::distr::sample_like("hcos", like, 34ULL);
auto dens = pluss::distr::pdf("hcos", x, {});
auto logprob = pluss::distr::logpdf("hcos", x, {});
auto dist = pluss::distr::cdf("hcos", x, {});
```

4.26 hyperbolicsecant

4.26.1 Signature

```
pluss::distr::sample("hyperbolicsecant", {}, sizes, cfg, seed);
pluss::distr::sample_like("hyperbolicsecant", {}, like, seed);
pluss::distr::pdf("hyperbolicsecant", x, {});
pluss::distr::logpdf("hyperbolicsecant", x, {});
pluss::distr::cdf("hyperbolicsecant", x, {});
```

4.26.2 Notes

Symmetric hyperbolic-secant family.

Aliases: `hsecant, hsec`.

Parameters: `{}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

4.26.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({-1.0, 0.0, 1.0}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("hsec", {}, {6}, cfg, 35ULL);
auto like_draws = pluss::distr::sample_like("hsec", like, 35ULL);
auto dens = pluss::distr::pdf("hsec", x, {});
auto logprob = pluss::distr::logpdf("hsec", x, {});
auto dist = pluss::distr::cdf("hsec", x, {});
```

4.27 hypergeometric

4.27.1 Signature

```
pluss::distr::sample("hypergeometric", {N, M, n}, sizes, cfg, seed);
pluss::distr::sample_like("hypergeometric", {N, M, n}, like, seed);
pluss::distr::pdf("hypergeometric", x, {N, M, n});
pluss::distr::logpdf("hypergeometric", x, {N, M, n});
pluss::distr::cdf("hypergeometric", x, {N, M, n});
```

4.27.2 Notes

Draws counts without replacement from a finite population.

Aliases: `hypergeom`.

Parameters: `{N, M, n}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

4.27.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({0.0, 1.0, 2.0}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("hypergeom", {20.0, 8.0, 5.0}, {6}, cfg, 36ULL);
auto like_draws = pluss::distr::sample_like("hypergeom", {20.0, 8.0, 5.0}, like, 36ULL);
auto pmf = pluss::distr::pdf("hypergeom", x, {20.0, 8.0, 5.0});
auto logprob = pluss::distr::logpdf("hypergeom", x, {20.0, 8.0, 5.0});
auto dist = pluss::distr::cdf("hypergeom", x, {20.0, 8.0, 5.0});
```

4.28 inversegaussian

4.28.1 Signature

```
pluss::distr::sample("inversegaussian", {mu, lambda}, sizes, cfg, seed);
pluss::distr::sample_like("inversegaussian", {mu, lambda}, like, seed);
pluss::distr::pdf("inversegaussian", x, {mu, lambda});
pluss::distr::logpdf("inversegaussian", x, {mu, lambda});
pluss::distr::cdf("inversegaussian", x, {mu, lambda});
```

4.28.2 Notes

Positive-support inverse Gaussian / Wald-type family with separate mean and shape.

Aliases: `ig, inversegauss, invgauss`.

Parameters: `{mu, lambda}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

4.28.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({0.5, 1.0, 2.0}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("ig", {1.5, 2.0}, {6}, cfg, 37ULL);
auto like_draws = pluss::distr::sample_like("ig", {1.5, 2.0}, like, 37ULL);
auto dens = pluss::distr::pdf("ig", x, {1.5, 2.0});
auto logprob = pluss::distr::logpdf("ig", x, {1.5, 2.0});
auto dist = pluss::distr::cdf("ig", x, {1.5, 2.0});
```

4.29 maxwell

4.29.1 Signature

```
pluss::distr::sample("maxwell", {scale}, sizes, cfg, seed);
pluss::distr::sample_like("maxwell", {scale}, like, seed);
pluss::distr::pdf("maxwell", x, {scale});
pluss::distr::logpdf("maxwell", x, {scale});
pluss::distr::cdf("maxwell", x, {scale});
```

4.29.2 Notes

Maxwell speed distribution.

Aliases: none.

Parameters: `{scale}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

4.29.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({0.5, 1.0, 2.0}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("maxwell", {1.0}, {6}, cfg, 38ULL);
auto like_draws = pluss::distr::sample_like("maxwell", {1.0}, like, 38ULL);
auto dens = pluss::distr::pdf("maxwell", x, {1.0});
auto logprob = pluss::distr::logpdf("maxwell", x, {1.0});
auto dist = pluss::distr::cdf("maxwell", x, {1.0});
```

4.30 nakagami

4.30.1 Signature

```
pluss::distr::sample("nakagami", {m, omega}, sizes, cfg, seed);
pluss::distr::sample_like("nakagami", {m, omega}, like, seed);
pluss::distr::pdf("nakagami", x, {m, omega});
pluss::distr::logpdf("nakagami", x, {m, omega});
pluss::distr::cdf("nakagami", x, {m, omega});
```

4.30.2 Notes

Positive-support Nakagami family.

Aliases: `nakagamim, nakagami-m`.

Parameters: `{m, omega}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

4.30.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({0.5, 1.0, 2.0}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("nakagami", {1.5, 2.0}, {6}, cfg, 39ULL);
auto like_draws = pluss::distr::sample_like("nakagami", {1.5, 2.0}, like, 39ULL);
auto dens = pluss::distr::pdf("nakagami", x, {1.5, 2.0});
auto logprob = pluss::distr::logpdf("nakagami", x, {1.5, 2.0});
auto dist = pluss::distr::cdf("nakagami", x, {1.5, 2.0});
```

4.31 negativebinomial

4.31.1 Signature

```
pluss::distr::sample("negativebinomial", {r, p}, sizes, cfg, seed);
pluss::distr::sample_like("negativebinomial", {r, p}, like, seed);
pluss::distr::pdf("negativebinomial", x, {r, p});
pluss::distr::logpdf("negativebinomial", x, {r, p});
pluss::distr::cdf("negativebinomial", x, {r, p});
```

4.31.2 Notes

Counts failures before a target number of successes under the implemented convention.

Aliases: `negbinomial, negbinom, pascal`.

Parameters: `{r, p}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

4.31.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({0.0, 1.0, 2.0, 3.0}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("pascal", {3.0, 0.45}, {6}, cfg, 40ULL);
auto like_draws = pluss::distr::sample_like("pascal", {3.0, 0.45}, like, 40ULL);
auto pmf = pluss::distr::pdf("pascal", x, {3.0, 0.45});
auto logprob = pluss::distr::logpdf("pascal", x, {3.0, 0.45});
auto dist = pluss::distr::cdf("pascal", x, {3.0, 0.45});
```

4.32 normaltrunc

4.32.1 Signature

```
pluss::distr::sample("normaltrunc", {a, b, mu, sigma}, sizes, cfg, seed);
pluss::distr::sample_like("normaltrunc", {a, b, mu, sigma}, like, seed);
pluss::distr::pdf("normaltrunc", x, {a, b, mu, sigma});
pluss::distr::logpdf("normaltrunc", x, {a, b, mu, sigma});
pluss::distr::cdf("normaltrunc", x, {a, b, mu, sigma});
```

4.32.2 Notes

Truncated normal clipped to `[a, b]` before normalization.

Aliases: `normaltruncated, gausstrunc`.

Parameters: `{a, b, mu, sigma}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

4.32.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({-1.0, 0.0, 1.0}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("normaltrunc", {-1.0, 1.0, 0.0, 1.0}, {6}, cfg, 41ULL);
auto like_draws = pluss::distr::sample_like("normaltrunc", {-1.0, 1.0, 0.0, 1.0}, like,␣
↪41ULL);
auto dens = pluss::distr::pdf("normaltrunc", x, {-1.0, 1.0, 0.0, 1.0});
auto logprob = pluss::distr::logpdf("normaltrunc", x, {-1.0, 1.0, 0.0, 1.0});
auto dist = pluss::distr::cdf("normaltrunc", x, {-1.0, 1.0, 0.0, 1.0});
```

4.33 planck

4.33.1 Signature

```
pluss::distr::sample("planck", {lambda, scale}, sizes, cfg, seed);
pluss::distr::sample_like("planck", {lambda, scale}, like, seed);
pluss::distr::pdf("planck", x, {lambda, scale});
pluss::distr::logpdf("planck", x, {lambda, scale});
pluss::distr::cdf("planck", x, {lambda, scale});
```

4.33.2 Notes

Discrete positive-count family with exponentially damped tails. Use it when you want a count distribution that stays strictly positive and decays faster than a pure power law. `lambda` sets the near-origin shape and `scale` controls the tail decay rate.

Aliases: none.

Parameters: `{lambda, scale}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

4.33.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({1.0, 2.0, 3.0, 4.0}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("planck", {2.0, 1.5}, {6}, cfg, 42ULL);
auto like_draws = pluss::distr::sample_like("planck", {2.0, 1.5}, like, 42ULL);
auto pmf = pluss::distr::pdf("planck", x, {2.0, 1.5});
auto logprob = pluss::distr::logpdf("planck", x, {2.0, 1.5});
auto dist = pluss::distr::cdf("planck", x, {2.0, 1.5});
```

4.34 poisson

4.34.1 Signature

```
pluss::distr::sample("poisson", {lambda}, sizes, cfg, seed);
pluss::distr::sample_like("poisson", {lambda}, like, seed);
pluss::distr::pdf("poisson", x, {lambda});
pluss::distr::logpdf("poisson", x, {lambda});
pluss::distr::cdf("poisson", x, {lambda});
```

4.34.2 Notes

Counts independent events occurring at rate `lambda`.

Aliases: `po`.

Parameters: `{lambda}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

4.34.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({0.0, 1.0, 2.0, 3.0}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("poisson", {2.0}, {6}, cfg, 43ULL);
auto like_draws = pluss::distr::sample_like("poisson", {2.0}, like, 43ULL);
auto pmf = pluss::distr::pdf("poisson", x, {2.0});
auto logprob = pluss::distr::logpdf("poisson", x, {2.0});
auto dist = pluss::distr::cdf("poisson", x, {2.0});
```

4.35 quadratic

4.35.1 Signature

```
pluss::distr::sample("quadratic", {}, sizes, cfg, seed);
pluss::distr::sample_like("quadratic", {}, like, seed);
pluss::distr::pdf("quadratic", x, {});
pluss::distr::logpdf("quadratic", x, {});
pluss::distr::cdf("quadratic", x, {});
```

4.35.2 Notes

Bounded quadratic family.

Aliases: `quad, quadr`.

Parameters: `{}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

4.35.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({-0.5, 0.0, 0.5}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("quadr", {}, {6}, cfg, 44ULL);
auto like_draws = pluss::distr::sample_like("quadr", like, 44ULL);
auto dens = pluss::distr::pdf("quadr", x, {});
auto logprob = pluss::distr::logpdf("quadr", x, {});
auto dist = pluss::distr::cdf("quadr", x, {});
```

4.36 rademacher

4.36.1 Signature

```
pluss::distr::sample("rademacher", {}, sizes, cfg, seed);
pluss::distr::sample_like("rademacher", {}, like, seed);
pluss::distr::pdf("rademacher", x, {});
pluss::distr::logpdf("rademacher", x, {});
pluss::distr::cdf("rademacher", x, {});
```

4.36.2 Notes

Produces symmetric -1/+1 sign draws.

Aliases: none.

Parameters: {}.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

4.36.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({-1.0, 1.0}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("rademacher", {}, {6}, cfg, 45ULL);
auto like_draws = pluss::distr::sample_like("rademacher", like, 45ULL);
auto pmf = pluss::distr::pdf("rademacher", x, {});
auto logprob = pluss::distr::logpdf("rademacher", x, {});
auto dist = pluss::distr::cdf("rademacher", x, {});
```

4.37 studentst

4.37.1 Signature

```
pluss::distr::sample("studentst", {nu}, sizes, cfg, seed);
pluss::distr::sample("studentst", {nu, chi, eta}, sizes, cfg, seed);
pluss::distr::sample_like("studentst", {nu}, like, seed);
pluss::distr::sample_like("studentst", {nu, chi, eta}, like, seed);
pluss::distr::pdf("studentst", x, {nu});
pluss::distr::pdf("studentst", x, {nu, chi, eta});
pluss::distr::logpdf("studentst", x, {nu});
pluss::distr::logpdf("studentst", x, {nu, chi, eta});
pluss::distr::cdf("studentst", x, {nu});
pluss::distr::cdf("studentst", x, {nu, chi, eta});
```

4.37.2 Notes

Student-t family with optional location/scale extension.

Aliases: `t`.

Parameters: `{nu} or {nu, chi, eta}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

4.37.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({-1.0, 0.0, 1.0}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("t", {5.0}, {6}, cfg, 46ULL);
auto like_draws = pluss::distr::sample_like("t", {5.0}, like, 46ULL);
auto dens = pluss::distr::pdf("t", x, {5.0});
auto logprob = pluss::distr::logpdf("t", x, {5.0});
auto dist = pluss::distr::cdf("t", x, {5.0});
```

4.38 triangular

4.38.1 Signature

```
pluss::distr::sample("triangular", {a, c, b}, sizes, cfg, seed);
pluss::distr::sample_like("triangular", {a, c, b}, like, seed);
pluss::distr::pdf("triangular", x, {a, c, b});
pluss::distr::logpdf("triangular", x, {a, c, b});
pluss::distr::cdf("triangular", x, {a, c, b});
```

4.38.2 Notes

Triangular family with lower bound `a`, mode `c`, and upper bound `b`.

Aliases: `tri`.

Parameters: `{a, c, b}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

4.38.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({0.0, 0.5, 1.0}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("tri", {0.0, 0.5, 1.0}, {6}, cfg, 47ULL);
auto like_draws = pluss::distr::sample_like("tri", {0.0, 0.5, 1.0}, like, 47ULL);
auto dens = pluss::distr::pdf("tri", x, {0.0, 0.5, 1.0});
auto logprob = pluss::distr::logpdf("tri", x, {0.0, 0.5, 1.0});
auto dist = pluss::distr::cdf("tri", x, {0.0, 0.5, 1.0});
```

4.39 pareto

4.39.1 Signature

```
pluss::distr::sample("pareto", {xm, alpha}, sizes, cfg, seed);
pluss::distr::sample_like("pareto", {xm, alpha}, like, seed);
pluss::distr::pdf("pareto", x, {xm, alpha});
pluss::distr::logpdf("pareto", x, {xm, alpha});
pluss::distr::cdf("pareto", x, {xm, alpha});
```

4.39.2 Notes

Classical Pareto tail family on values at or above the scale threshold.

Aliases: none.

Parameters: `{xm, alpha}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

4.39.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({1.0, 2.0, 4.0}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("pareto", {1.0, 3.0}, {6}, cfg, 48ULL);
```

(continues on next page)

(continued from previous page)

```
auto like_draws = pluss::distr::sample_like("pareto", {1.0, 3.0}, like, 48ULL);
auto dens = pluss::distr::pdf("pareto", x, {1.0, 3.0});
auto logprob = pluss::distr::logpdf("pareto", x, {1.0, 3.0});
auto dist = pluss::distr::cdf("pareto", x, {1.0, 3.0});
```

4.40 lomax

4.40.1 Signature

```
pluss::distr::sample("lomax", {scale, alpha}, sizes, cfg, seed);
pluss::distr::sample_like("lomax", {scale, alpha}, like, seed);
pluss::distr::pdf("lomax", x, {scale, alpha});
pluss::distr::logpdf("lomax", x, {scale, alpha});
pluss::distr::cdf("lomax", x, {scale, alpha});
```

4.40.2 Notes

Shifted Pareto / Lomax family.

Aliases: `pareto2`.

Parameters: `{scale, alpha}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

4.40.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({0.5, 1.0, 2.0}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("lomax", {2.0, 3.0}, {6}, cfg, 49ULL);
auto like_draws = pluss::distr::sample_like("lomax", {2.0, 3.0}, like, 49ULL);
auto dens = pluss::distr::pdf("lomax", x, {2.0, 3.0});
auto logprob = pluss::distr::logpdf("lomax", x, {2.0, 3.0});
auto dist = pluss::distr::cdf("lomax", x, {2.0, 3.0});
```

4.41 rayleigh

4.41.1 Signature

```
pluss::distr::sample("rayleigh", {scale}, sizes, cfg, seed);
pluss::distr::sample_like("rayleigh", {scale}, like, seed);
pluss::distr::pdf("rayleigh", x, {scale});
pluss::distr::logpdf("rayleigh", x, {scale});
pluss::distr::cdf("rayleigh", x, {scale});
```

4.41.2 Notes

Radial magnitude family derived from isotropic Gaussian components.

Aliases: `rayl`.

Parameters: `{scale}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

4.41.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({0.5, 1.0, 2.0}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("rayl", {1.0}, {6}, cfg, 50ULL);
auto like_draws = pluss::distr::sample_like("rayl", {1.0}, like, 50ULL);
auto dens = pluss::distr::pdf("rayl", x, {1.0});
auto logprob = pluss::distr::logpdf("rayl", x, {1.0});
auto dist = pluss::distr::cdf("rayl", x, {1.0});
```

4.42 `rice`

4.42.1 Signature

```
pluss::distr::sample("rice", {nu, sigma}, sizes, cfg, seed);
pluss::distr::sample_like("rice", {nu, sigma}, like, seed);
pluss::distr::pdf("rice", x, {nu, sigma});
pluss::distr::logpdf("rice", x, {nu, sigma});
pluss::distr::cdf("rice", x, {nu, sigma});
```

4.42.2 Notes

Non-central magnitude family often used in signal models.

Aliases: `rician`.

Parameters: `{nu, sigma}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

4.42.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({0.5, 1.5, 2.5}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("rice", {1.0, 1.0}, {6}, cfg, 51ULL);
```

(continues on next page)

(continued from previous page)

```
auto like_draws = pluss::distr::sample_like("rice", {1.0, 1.0}, like, 51ULL);
auto dens = pluss::distr::pdf("rice", x, {1.0, 1.0});
auto logprob = pluss::distr::logpdf("rice", x, {1.0, 1.0});
auto dist = pluss::distr::cdf("rice", x, {1.0, 1.0});
```

4.43 semicircle

4.43.1 Signature

```
pluss::distr::sample("semicircle", {mu, radius}, sizes, cfg, seed);
pluss::distr::sample_like("semicircle", {mu, radius}, like, seed);
pluss::distr::pdf("semicircle", x, {mu, radius});
pluss::distr::logpdf("semicircle", x, {mu, radius});
pluss::distr::cdf("semicircle", x, {mu, radius});
```

4.43.2 Notes

Compact-support semicircle family.

Aliases: `semicirc, wigner`.

Parameters: `{mu, radius}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

4.43.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({-1.0, 0.0, 1.0}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("semicirc", {0.0, 1.0}, {6}, cfg, 52ULL);
auto like_draws = pluss::distr::sample_like("semicirc", {0.0, 1.0}, like, 52ULL);
auto dens = pluss::distr::pdf("semicirc", x, {0.0, 1.0});
auto logprob = pluss::distr::logpdf("semicirc", x, {0.0, 1.0});
auto dist = pluss::distr::cdf("semicirc", x, {0.0, 1.0});
```

4.44 skellam

4.44.1 Signature

```
pluss::distr::sample("skellam", {mu1, mu2}, sizes, cfg, seed);
pluss::distr::sample_like("skellam", {mu1, mu2}, like, seed);
pluss::distr::pdf("skellam", x, {mu1, mu2});
pluss::distr::logpdf("skellam", x, {mu1, mu2});
pluss::distr::cdf("skellam", x, {mu1, mu2});
```

4.44.2 Notes

Difference of two independent Poisson counts.

Aliases: none.

Parameters: `{mu1, mu2}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

4.44.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({-1.0, 0.0, 1.0}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("skellam", {2.0, 1.0}, {6}, cfg, 53ULL);
auto like_draws = pluss::distr::sample_like("skellam", {2.0, 1.0}, like, 53ULL);
auto pmf = pluss::distr::pdf("skellam", x, {2.0, 1.0});
auto logprob = pluss::distr::logpdf("skellam", x, {2.0, 1.0});
auto dist = pluss::distr::cdf("skellam", x, {2.0, 1.0});
```

4.45 tukeylambda

4.45.1 Signature

```
pluss::distr::sample("tukeylambda", {lambda}, sizes, cfg, seed);
pluss::distr::sample_like("tukeylambda", {lambda}, like, seed);
pluss::distr::pdf("tukeylambda", x, {lambda});
pluss::distr::logpdf("tukeylambda", x, {lambda});
pluss::distr::cdf("tukeylambda", x, {lambda});
```

4.45.2 Notes

Flexible shape family controlled by the lambda tail parameter.

Aliases: none.

Parameters: `{lambda}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

4.45.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({-1.0, 0.0, 1.0}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("tukeylambda", {0.14}, {6}, cfg, 54ULL);
```

(continues on next page)

(continued from previous page)

```
auto like_draws = pluss::distr::sample_like("tukeylambda", {0.14}, like, 54ULL);
auto dens = pluss::distr::pdf("tukeylambda", x, {0.14});
auto logprob = pluss::distr::logpdf("tukeylambda", x, {0.14});
auto dist = pluss::distr::cdf("tukeylambda", x, {0.14});
```

4.46 ushape

4.46.1 Signature

```
pluss::distr::sample("ushape", {}, sizes, cfg, seed);
pluss::distr::sample_like("ushape", {}, like, seed);
pluss::distr::pdf("ushape", x, {});
pluss::distr::logpdf("ushape", x, {});
pluss::distr::cdf("ushape", x, {});
```

4.46.2 Notes

Symmetric U-shaped bounded family.

Aliases: `u`.

Parameters: `{}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

4.46.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({0.1, 0.5, 0.9}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("u", {}, {6}, cfg, 55ULL);
auto like_draws = pluss::distr::sample_like("u", like, 55ULL);
auto dens = pluss::distr::pdf("u", x, {});
auto logprob = pluss::distr::logpdf("u", x, {});
auto dist = pluss::distr::cdf("u", x, {});
```

4.47 vonmises

4.47.1 Signature

```
pluss::distr::sample("vonmises", {mu, kappa}, sizes, cfg, seed);
pluss::distr::sample_like("vonmises", {mu, kappa}, like, seed);
pluss::distr::pdf("vonmises", x, {mu, kappa});
pluss::distr::logpdf("vonmises", x, {mu, kappa});
pluss::distr::cdf("vonmises", x, {mu, kappa});
```

4.47.2 Notes

Circular distribution on angles.

Aliases: none.

Parameters: `{mu, kappa}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

4.47.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({-1.0, 0.0, 1.0}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("vonmises", {0.0, 2.0}, {6}, cfg, 56ULL);
auto like_draws = pluss::distr::sample_like("vonmises", {0.0, 2.0}, like, 56ULL);
auto dens = pluss::distr::pdf("vonmises", x, {0.0, 2.0});
auto logprob = pluss::distr::logpdf("vonmises", x, {0.0, 2.0});
auto dist = pluss::distr::cdf("vonmises", x, {0.0, 2.0});
```

4.48 wald

4.48.1 Signature

```
pluss::distr::sample("wald", {mu}, sizes, cfg, seed);
pluss::distr::sample_like("wald", {mu}, like, seed);
pluss::distr::pdf("wald", x, {mu});
pluss::distr::logpdf("wald", x, {mu});
pluss::distr::cdf("wald", x, {mu});
```

4.48.2 Notes

Single-parameter Wald special case exposed by the runtime.

Aliases: none.

Parameters: `{mu}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

4.48.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({0.5, 1.0, 2.0}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("wald", {2.0}, {6}, cfg, 57ULL);
```

(continues on next page)

(continued from previous page)

```
auto like_draws = pluss::distr::sample_like("wald", {2.0}, like, 57ULL);
auto dens = pluss::distr::pdf("wald", x, {2.0});
auto logprob = pluss::distr::logpdf("wald", x, {2.0});
auto dist = pluss::distr::cdf("wald", x, {2.0});
```

4.49 weibull

4.49.1 Signature

```
pluss::distr::sample("weibull", {shape, scale}, sizes, cfg, seed);
pluss::distr::sample("weibull", {theta, alpha, beta}, sizes, cfg, seed);
pluss::distr::sample_like("weibull", {shape, scale}, like, seed);
pluss::distr::sample_like("weibull", {theta, alpha, beta}, like, seed);
pluss::distr::pdf("weibull", x, {shape, scale});
pluss::distr::pdf("weibull", x, {theta, alpha, beta});
pluss::distr::logpdf("weibull", x, {shape, scale});
pluss::distr::logpdf("weibull", x, {theta, alpha, beta});
pluss::distr::cdf("weibull", x, {shape, scale});
pluss::distr::cdf("weibull", x, {theta, alpha, beta});
```

4.49.2 Notes

Supports both the short two-parameter Weibull form and the legacy three-parameter location-scale-shape form.

Aliases: `wbl`, `frechet`.

Parameters: `{shape, scale}` or `{theta, alpha, beta}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

4.49.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({0.5, 1.0, 2.0}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("weibull", {0.0, 2.0, 1.5}, {6}, cfg, 58ULL);
auto like_draws = pluss::distr::sample_like("weibull", {0.0, 2.0, 1.5}, like, 58ULL);
auto dens = pluss::distr::pdf("weibull", x, {0.0, 2.0, 1.5});
auto logprob = pluss::distr::logpdf("weibull", x, {0.0, 2.0, 1.5});
auto dist = pluss::distr::cdf("weibull", x, {0.0, 2.0, 1.5});
```

4.50 yule

4.50.1 Signature

```
pluss::distr::sample("yule", {rho}, sizes, cfg, seed);
pluss::distr::sample_like("yule", {rho}, like, seed);
pluss::distr::pdf("yule", x, {rho});
pluss::distr::logpdf("yule", x, {rho});
pluss::distr::cdf("yule", x, {rho});
```

4.50.2 Notes

Heavy-tailed discrete Yule-Simon family.

Aliases: `yulesimon`.

Parameters: `{rho}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

4.50.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({1.0, 2.0, 3.0}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("yule", {3.0}, {6}, cfg, 59ULL);
auto like_draws = pluss::distr::sample_like("yule", {3.0}, like, 59ULL);
auto pmf = pluss::distr::pdf("yule", x, {3.0});
auto logprob = pluss::distr::logpdf("yule", x, {3.0});
auto dist = pluss::distr::cdf("yule", x, {3.0});
```

4.51 zeta

4.51.1 Signature

```
pluss::distr::sample("zeta", {s}, sizes, cfg, seed);
pluss::distr::sample_like("zeta", {s}, like, seed);
pluss::distr::pdf("zeta", x, {s});
pluss::distr::logpdf("zeta", x, {s});
pluss::distr::cdf("zeta", x, {s});
```

4.51.2 Notes

Zipf/Zeta count family. Use values at or above 1 for the mass function.

Aliases: `zipf`.

Parameters: `{s}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

4.51.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({1.0, 2.0, 3.0}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("zipf", {2.5}, {6}, cfg, 60ULL);
auto like_draws = pluss::distr::sample_like("zipf", {2.5}, like, 60ULL);
auto pmf = pluss::distr::pdf("zipf", x, {2.5});
auto logprob = pluss::distr::logpdf("zipf", x, {2.5});
auto dist = pluss::distr::cdf("zipf", x, {2.5});
```

4.52 gig

4.52.1 Signature

```
pluss::distr::sample("gig", {lambda, chi, psi}, sizes, cfg, seed);
pluss::distr::sample_like("gig", {lambda, chi, psi}, like, seed);
pluss::distr::pdf("gig", x, {lambda, chi, psi});
pluss::distr::logpdf("gig", x, {lambda, chi, psi});
pluss::distr::cdf("gig", x, {lambda, chi, psi});
```

4.52.2 Notes

Generalized inverse Gaussian family on the positive real line. `lambda` selects the subfamily, while `chi` and `psi` control the inverse-Gaussian versus gamma-like pull on the tails. In practice this is the workhorse mixing law behind several heavier-tailed continuous models, including the generalized hyperbolic family.

Aliases: none.

Parameters: `{lambda, chi, psi}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

4.52.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({0.5, 1.0, 2.0}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("gig", {0.5, 1.0, 2.0}, {6}, cfg, 61ULL);
auto like_draws = pluss::distr::sample_like("gig", {0.5, 1.0, 2.0}, like, 61ULL);
auto dens = pluss::distr::pdf("gig", x, {0.5, 1.0, 2.0});
auto logprob = pluss::distr::logpdf("gig", x, {0.5, 1.0, 2.0});
auto dist = pluss::distr::cdf("gig", x, {0.5, 1.0, 2.0});
```

4.53 gh

4.53.1 Signature

```
pluss::distr::sample("gh", {lambda, alpha, beta, mu, delta}, sizes, cfg, seed);
pluss::distr::sample_like("gh", {lambda, alpha, beta, mu, delta}, like, seed);
pluss::distr::pdf("gh", x, {lambda, alpha, beta, mu, delta});
pluss::distr::logpdf("gh", x, {lambda, alpha, beta, mu, delta});
pluss::distr::cdf("gh", x, {lambda, alpha, beta, mu, delta});
```

4.53.2 Notes

Five-parameter generalized hyperbolic family for skewed, heavy-tailed continuous data. `lambda` chooses the subclass, `alpha` and `beta` control tail weight and asymmetry, and `mu` / `delta` provide location and scale. This is the flexible parent family behind special cases like NIG.

Aliases: none.

Parameters: `{lambda, alpha, beta, mu, delta}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

4.53.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({-1.0, 0.0, 1.0}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("gh", {1.0, 2.0, 0.5, 0.0, 1.0}, {6}, cfg, 62ULL);
auto like_draws = pluss::distr::sample_like("gh", {1.0, 2.0, 0.5, 0.0, 1.0}, like,␣
↪62ULL);
auto dens = pluss::distr::pdf("gh", x, {1.0, 2.0, 0.5, 0.0, 1.0});
auto logprob = pluss::distr::logpdf("gh", x, {1.0, 2.0, 0.5, 0.0, 1.0});
auto dist = pluss::distr::cdf("gh", x, {1.0, 2.0, 0.5, 0.0, 1.0});
```

4.54 nig

4.54.1 Signature

```
pluss::distr::sample("nig", {alpha, beta, mu, delta}, sizes, cfg, seed);
pluss::distr::sample_like("nig", {alpha, beta, mu, delta}, like, seed);
pluss::distr::pdf("nig", x, {alpha, beta, mu, delta});
pluss::distr::logpdf("nig", x, {alpha, beta, mu, delta});
pluss::distr::cdf("nig", x, {alpha, beta, mu, delta});
```

4.54.2 Notes

Normal-inverse-Gaussian special case of the generalized hyperbolic family, with `alpha > |beta|` required by the runtime. Use it when you need a continuous law that can stay much more tail-heavy and skewed than a Gaussian while still supporting direct sampling, density evaluation, and object-based cumulant generation through `pluss::distr::Nig`.

Aliases: none.

Parameters: `{alpha, beta, mu, delta}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

4.54.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({-1.0, 0.0, 1.0}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("nig", {2.0, 0.5, 0.0, 1.0}, {6}, cfg, 63ULL);
auto like_draws = pluss::distr::sample_like("nig", {2.0, 0.5, 0.0, 1.0}, like, 63ULL);
auto dens = pluss::distr::pdf("nig", x, {2.0, 0.5, 0.0, 1.0});
auto logprob = pluss::distr::logpdf("nig", x, {2.0, 0.5, 0.0, 1.0});
auto dist = pluss::distr::cdf("nig", x, {2.0, 0.5, 0.0, 1.0});
```

CHAPTER

FIVE

BUILD OUTPUTS

5.1 Install Doc Dependencies

```
python3 -m pip install -r requirements.txt
```

5.2 Build HTML

```
make html
```

The HTML output lands in:

```
_build/html
```

Expected entry files:

- `_build/html/index.html`
- `_build/html/quickstart.html`
- `_build/html/api_reference.html`
- `_build/html/object_api_reference.html`
- `_build/html/distribution_catalog.html`
- `_build/html/build_outputs.html`

5.3 Build PDF

```
make pdf
```

This expects a working LaTeX toolchain with `latexmk`.

Generated files:

- `_build/latex/plussma_distribution_docs.tex`
- `_build/latex/plussma_distribution_docs.pdf`

The final PDF manual is:

```
_build/latex/plussma_distribution_docs.pdf
```

5.4 Current Outputs

The current checked build produced:

- HTML site root: `_build/html/index.html`
- PDF manual: `_build/latex/plussma_distribution_docs.pdf`

5.5 Relevant Code Artifacts

The docs describe the production surface implemented in:

- `PlussMa/Stats/distributions/distr.cpp`
- `PlussMa/Stats/distributions/distr_objects_continuous.cpp`
- `PlussMa/Stats/distributions/distr_objects_discrete.cpp`
- `PlussMa/Stats/distributions/distr_objects_special.cpp`
- `PlussMa/include/pluss/subsp/stat/distr_fwd.h`
- `PlussMa/include/pluss/subsp/stat/distr.h`
- `PlussMa/include/pluss/ma/stats.h`
- `PlussMa/examples/Numerical/distributions_demo.cpp`
- `PlussMa/Stats/distributions/examples/distributions_objects_demo.cpp`

5.6 Validation Path

The runtime coverage for the documented surface is in:

```
PlussMa/Stats/distributions/test/distr_test.cpp
```